PROPOSAL FOR

CLASS PARTICIPATION IN FUNCTIONS

FOR TECHNICAL DEBT ANALYSIS OF

PROCEDURAL AND OBJECT ORIENTED LANGUAGES

by

Grant Jared Nelson

A dissertation submitted in partial fulfillment
of the requirements for the degree

of

Doctor of Philosophy

in

Computer Science

MONTANA STATE UNIVERSITY
Bozeman, Montana

December 2025

©COPYRIGHT

by

Grant Jared Nelson

2025

All Rights Reserved

ACKNOWLEDGEMENTS

I would like to thank Dr. Clemente Izurieta and Dr. Derek Reimanis for their valuable review, feedback, and support for this report. This report and associated research were done without any funding source and are not under any grant. No AI assistance was used when writing this paper. Special thanks to my family for their patience and encouragement.

TABLE OF CONTENTS

1. INTRODUCTION 1

Research Problem 3
Hypothesis 4
Motivation 4

2. BACKGROUND 6

Related Works 6
Potpourri Modules 6
GodExpo 9
Sarker, Jubaer, Shohrawardi, Das, and Siddik 13
CompreGo 15
God Classes 17
Fuzzy Sets 19
Design Recovery and Migration 21
Belady and Evanglisti 22
Hutchens and Basili 23
Biggerstaff 25
Liu and Wilde 26
Livadas and Johnson 28
Sneed and Nyáry 29
Cimitile, De Lucia, Di Lucca, and Fasolino 29
Design Transformations 31
Zou and Kontogiannis 32
Hossain and Zulkarnine 33
Siddik, Gias, and Khaled 34
Selim, Siddik, Gias, Abdullah-Al-Wadud, and Shah 36
Siddik, Gias, Selim, Khaled, and Sakib 37
Siddik, Selim, Gias, and Khaled 37
Islam, Toma, Selim, Gias, and Khaled 38
Samsuddoha, Siddik, Selim, and Khaled 41
Membership and Participation 41
Significance of the Study 45

3. APPROACH 47

Determine Feature Description 48
Java Membership 49
Java Participation 51

TABLE OF CONTENTS – CONTINUED

Identify Variance in Java 53
Participation in Go 55
Analyze Go Participation 56
Report Findings 57
Future Work 57

4. PRELIMINARY WORK AND RESULTS 60

Feature Definition 60
Go Feature Abstractor 62
Java Feature Abstractor 63
Design Recovery and Technical Debt Analysis 64

5. TIMELINE 65

4th Quarter 2025 65
1st Quarter 2026 65
2nd Quarter 2026 66
3rd Quarter 2026 66
4th Quarter 2026 67

REFERENCES CITED 68

LIST OF FIGURES

Figure Page

1.1 SonarQube rule count by language 2

2.1 Example Degenerate C-Partition Clusters 21

3.1 Software Pipeline Diagram 47

NOMENCLATURE

AST	Abstract Syntax Tree
$ATFD$	Access To Foreign Data
$\mathcal{C}$	All classes and objects in a project
CC	Cyclomatic Complexity
CI/CD	Continuous integration and continuous delivery
DD	Design Debt
DR	Design Recovery
$GBOI$	Global-Based Object Identification
GC	God Class
$\mathcal{F}$	All functions and methods in a project
LOC	Lines of Code
M	Membership Matrix
MSE	Mean Squared Error
OO	Object-Oriented Languages
$\mathcal{P}_c$	Non-Degenerate Hard C-Partition Space
$\mathcal{P}_{co}$	Degenerate Hard C-Partition Partition
$\mathcal{P}_{fc}$	Non-Degenerate Fuzzy C-Partition Partition
$\mathcal{P}_{fo}$	Degenerate Fuzzy C-Partition Partition
P	Participation Matrix
PCC	Pearson Correlation Coefficient
PL	Procedural Languages
$RBOI$	Receiver-Based Object Identification
$TBOI$	Type-Based Object Identification
TCC	Tight Class Cohesion
TD	Technical Debt
TDD	Technical Debt Dataset
V_{cn}	Set of All $c \times n$ Matrices
WMC	Weighted Method Count

ABSTRACT

Technical Debt (TD) causes an increase in the effort required by developers to add new features and perform maintenance [1]. Several tools have been written to help identify, prioritize, and reduce TD by running TD analysis tools on a project's source code. The TD analysis algorithms are based on metrics, such as those described by Marinescu [2], that require knowledge of the membership between a method and a class. Procedural languages (PL), such as Go, do not define a membership like Object Oriented languages (OO) do, therefore PL can not currently have the same analysis run on them.

To support practitioners who choose to use PL, we propose a way to estimate membership using Design Recovery (DR) algorithms. This fuzzy estimate describes how objects participate in a function. Our research aims to determine whether DR can produce a participation that is as useful as membership when performing TD analysis. This paper outlines the background and process for performing our research.

Our research compares the membership from Java source code against participation derived from the same Java source code as if the membership was not fully defined. This will determine whether participation can be used in place of membership. It also helps us identify the cause of any differences between the estimated membership and the actual membership.

Using the same DR algorithm, we will create a tool for producing the participation given Go source code. Using this tool, we will perform empirical tests with several companies to determine if we can provide a useful TD analysis of Go.

The results of the Java comparison, a description of the selected DR algorithm, and the results of the empirical test will be written in a follow up report.

1

INTRODUCTION

The computer science industry is rapidly growing as the demand increases for more software applications in every part of our lives. To meet the demand, companies will create large teams of developers, utilize modern technology, reuse legacy code, leverage generative AI agents, and set demanding deadlines. When developers intentionally or unintentionally sacrifice quality to meet a deadline, the Technical Debt (TD) increases in a code base. TD causes an increase in the effort required by developers to add new features and perform maintenance [3].

TD analysis tools can automatically identify, rank, and prioritize potential TD issues in a code base. Many companies use programmatic TD analysis tools to help reduce TD. The reduction of TD measurably reduces the time and cost of delivering software and features [3, 4, 5, 6]. Some companies integrate TD analysis tools into their CI/CD or *commit* actions.

Unfortunately, Procedural Languages (PL) do not have as many TD analysis tools as Object-Oriented Languages (OO) [7]. Currently, programmatic TD analysis tools cannot perform the same Design Debt (DD) analysis for PL. This is due to PL having different structure of code compared to OO. DD is a specific form of TD which focuses on the structure of the code [5] and compares those structures against known software patterns, smells, and anti-patterns. Since PL have a different structure than OO, the DD analysis currently cannot be directly run on PL programs.

Some TD analysis tools, such as CodeScene[1] and SonarQube[2], do provide analysis options for PL. However, their support is limited compared to other languages [7]. Therefore,

[1] `https://codescene.com`
[2] `https://www.sonarsource.com/products/sonarqube`

practitioners who use PL, such as Go[3], do not have the same quality of DD analysis as when using OO. Figure 1.1 shows the SonarQube rule count by language[4]. Each rule is a type of evaluation of the source code to identify a vulnerability, a bug, a potential security concern, or a code smell. SonarQube defines each rule in detail with example code snippets for the corresponding language. We did not performing any evaluating of the individual rules. The rule count is the total number of rules for each language grouped by the rule type. Some of the rules perform some kind of TD analysis, e.g. rule RSPEC-1541[5] checks that the McCabe's Cyclomatic Complexity [8] of a Java method does not exceed a threshold. Figure 1.1 is an updated version of the figure given in our prior work [7].

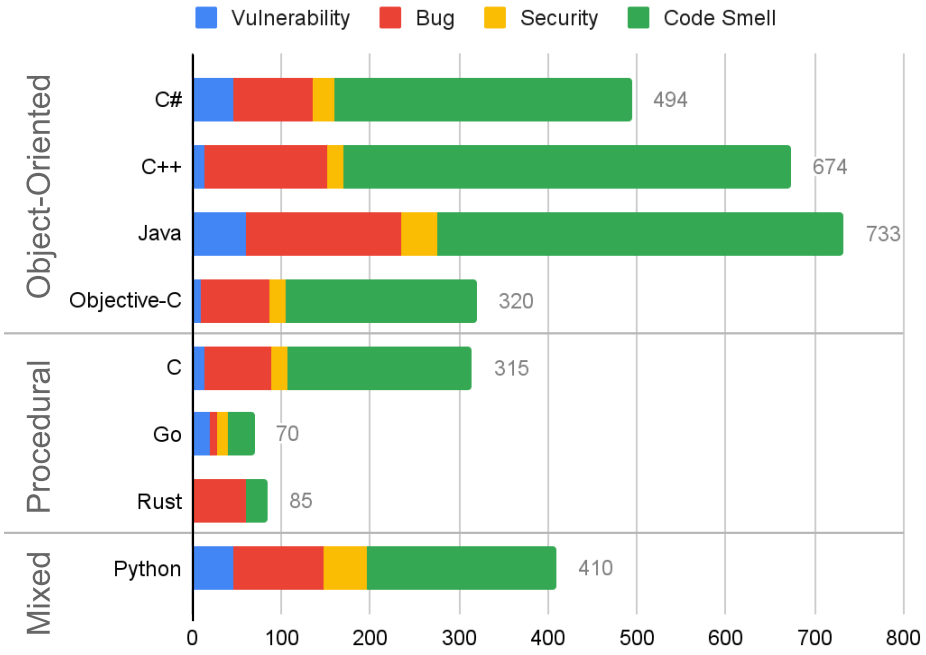


Figure 1.1: SonarQube rule count by language

[3] `https://golang.com`
[4] SonarQube rule count sourced from `https://rules.sonarsource.com` on Nov 15, 2025
[5] `https://rules.sonarsource.com/java/RSPEC-1541/`

This paper proposes an investigation of a novel metric that we call participation. Participation is a fuzzy estimate of membership that can be determined when membership is not explicitly defined. The investigation focuses on using participation for programmatic TD analysis of a PL project to provide a similar analysis to the existing TD analysis performed on an OO project.

Research Problem

OO define classes with methods. Each method has a membership with one class. This membership is used for metrics such as Chidamber and Kemerer's Weighted Method Count (WMC) [9], as discussed in the God Classes section. Membership is also used in other metrics when performing pattern and anti-pattern analysis. Consider membership as a way for developers to indicate that a method belongs completely to one class and not to any other class. Therefore, metrics, such as the McCabe's Cyclomatic Complexity (CC) [8] of a method, are attributed 100% to one class and 0% to all other classes.

Most PL do not specifically define a function's membership to an object in the same way that OO define a method's membership to a class. Without clear membership, some other metric must be used for TD analysis.

For this paper, *objects* and *classes* are nearly identical unless otherwise defined, same with *functions* and *methods*. This investigation and Design Recovery (DR) blurs any distinction between those terms since they are fundamentally the same and only different in the language paradigms. We try to use *objects* and *functions* when discussing PL, and *classes* and *methods* when discussing OO.

A function in a PL might not have a direct membership to an object, but it does have relationships to objects, data, and other functions. The objects participate in the function's purpose, so it may be possible to use this participation as a kind of fractional membership to provide a probabilistic metric. This allows the metrics to be weighted with that probability

when they are attributed to an object. The function to object relationship is then a fuzzy estimate describing the participation of the object in the function. Therefore, a function will have a fuzzy set of objects, the participation score, for that function.

Since methods in an OO can also have multiple classes participate, a participation score can be determined for OO methods as well. In the case that a method only has participation from one class, by definition that class is a full member in the fuzzy set. This means that we can compare membership directly against participation using the source code of an OO project.

Hypothesis

Participation, used as a generalized membership, can produce similar technical debt analysis results in object-oriented languages. A comparable form of participation can be defined for procedural languages to support similar technical debt analysis.

Motivation

I have worked for over twelve years as a software engineer at a large SaaS (software as a service) company. While there, the teams that I was on designed, implemented, and tested several very large applications. The teams continued to add features, maintain, and refactor the same applications for over ten years. Throughout this process the other developers and I could feel a build up of TD slowing progress. The teams could feel how refactors improved the product by paying down some of the built up TD.

The company I worked for tried using SonarQube, CodeScene, and a few other analysis tools to help identify and prioritize the TD that should be taken care of. None of these tools were useful in this endeavor other than to fix minor problems. Other teams in the same company working on different applications with similar complexity found the analysis tools

very useful. The main difference between the applications was the languages the teams used. The other teams' applications were built in OO and my teams' applications were built in a PL, Go.

This motivated us to research TD while relating it to PL. We noticed that many of the metrics used in TD analysis requires class and method memberships. We felt that similar TD analysis results could be produced if we could find a way to replace membership with another metric derived from PL.

BACKGROUND

The background first describes all related works. Then we define concepts that are then tied together by our prior work. The concepts are a well studied TD analytic called God Classes (GC), a basis for hard and fuzzy matrices used for clustering, and Design Recovery (DR) algorithms for associating data and functions in a program. Once these three concepts are defined, the membership and participation matrices that are the foundation for our research are described.

Related Works

There are a few prior work directly related to applying TD analysis designed for OO to PL. These works are discussed in this section. However, most of the similar related works are DR algorithms used to assist manual conversion from PL to OO. Some DR papers, such as Siddik, Gias, and Khaled, use TD metrics, such as Bieman and Kang's [10] class cohesion; however, they do not directly discuss leveraging those metrics to detect TD in PL. Those works are discussed in the section Design Recovery and Migration.

Potpourri Modules

Calliss and Cornelius (1990)[11] discuss a method for finding modules in PL and structural languages, e.g. Ada and Modula-2, that have taken on too much responsibility. They based the detection on the criteria for decomposing a system into modules, as outlined by Parnas (1972)[12], where each module should be assigned one responsibility or service for a program. The goal is to be able to hide design decisions inside a module and behind an interface so that the module can be tested, refactored, and modified in isolation. The interface between modules is a conceptual interface, not an OO interface as described by Liskov and Wing (1994)[13]. A "potpourri module" is a module that can be considered to

violate that criterion by taking on more than one responsibility.

Their work preceded Bieman and Kang (1995)[10], Chidamber and Kemerer (1994)[9], and Marinescu (2004)[2]. However, their work has some similarities to Bieman and Kang's Tight Class Cohesion (TCC), as discussed in the God Classes section, and the interconnections Belady and Evanglisti (1981)[14] used for DR, as discussed in the Belady and Evanglisti subsection.

They describe how to create an interconnection graph based on dependencies in the program. For example, a function (routine) that assigns or references a global variable is dependent on that variable. It is also dependent on the types in the parameters and results of the function's signature, and the types of variables that it defines internally. A global variable is dependent on its type and a module defines all entities within it. Each program entity is a node in the interconnection graph labeled with the entity kind (e.g. function, type, variable, constant), its declared name, and module. For each type of dependency, add a labeled directed edge into the graph. The label allows different types (e.g. defines, writes, references, of-type, result-of-type, parameter-of-type) of dependencies between the same two named entities.

If there are n modules then the interconnection graph has at minimum $n+1$ subgraphs. All the nodes that are defined for a module and the edges between those nodes is an entity-to-entity graph for one module. The edges that go between nodes of different module graphs define an entity-to-module graph.

The entity-to-entity graph for a module can be one or more disconnected graphs. Each disconnected graph gets categorized based on the kinds of entities in the graph. They define four sets of categories. The first category is a *collection of definitions* made up of only variables and types. The second category is a *group of related program units* made up of any entity kind except for variables and types. The third category is an *abstract data type* (not the same as an OO abstract data type) made up of any entity kind except variables and has

at least one type or function. The fourth category is an *abstract state machine* made up of any entities with at least one function.

If there is only one disconnected graph in a module, then that module only provides one service and therefore is not a potpourri module. If there are more than one category in a module, then that module is a potpourri module. If the disconnected graphs are all in the same category and that category is either an abstract data type or an abstract state machine, then it is a potpourri module. For example, a few disconnected graphs categorize as related program units could be a collection of mathematical functions, where as two disconnected abstract data types could be a stack definition and a separate queue definition in the same module. The stack and queue definitions should be in two separate modules to keep from violating the one responsibility criteria.

Taking this idea into an OO context, the entity-to-entity graph is similar to the idea of class cohesion as defined by Bieman and Kang [10], since it defines how global entities in a module depend on each other. If a module was considered to be a class, the global entities would be its fields, methods, and subclasses. Two methods in a class are directly connected when they both access the same field, mirroring how two functions in a module are directly connected when they both access the same variable. When two methods access the same variable then they belong to the same connected graph. When disconnected graphs exist in a class then the class has low cohesion since the fields and functions in the graphs could have been put into separate classes without them accessing each other's fields directly.

Calliss and Cornelius's research is similar to our research since they define a way to find an equivalent to a class that likely needs to be broken up. However, the disconnected graph determination doesn't take into account shared variables that could be accessed via an accessor instead of directly. TCC is a ratio that performs similar analysis whilst still allowing for single interconnection graph caused by a minimal number of shared variables.

The entity-to-entity disconnected graphs determine related variables, types, and

functions essentially defining classes and their members similar to DR. However, it assumes that each module is one or more class, meaning it would not be able to determine a coherent class spread across several modules. For example, if some of the functions for the class design were put into separate modules from the variables and types thus creating one collection of definitions module and one related program units module.

GodExpo

Yasir, Asad, Galib, Ganguly, and Siddik (2019)[15] describe GodExpo[1]. GodExpo is a tool that they claim can detect GCs in Go. This is similar to our research since our research could be used to detect GCs in PLs such as Go. However, our research includes performing DR on PL programs to create an estimation of membership, participation, between functions and objects that can be used when membership is unavailable for static analysis. Where as GodExpo only uses functions with receivers to estimate GCs whilst ignoring any function without a structure receiver and ignoring types without an underlying type of a structure. Since it is possible to write Go programs and never use a receiver on any function, GoExpo will return false negatives for some Go projects. It also could return some false positives because of a few bugs in their code.

They use Marinescu's method for detecting GCs [2]. However, they define a GC as "a class that threatens maintainability and understandability of code by performing most of the work alone." This definition seems contrary to their results where they found more than one GC in the two applications they ran GodExpo on. The two applications are Hugo[2] and Mattermost[3]. GodExpo detected seven GCs in Hugo v0.50 and 60 GCs in Mattermost v5.6.0. They also ran it on earlier versions of those two projects. Without proving the detected structures are GCs, they claimed "[GodExpo] has successfully detected God Structs

[1] `https://github.com/rafed/GodExpo`
[2] `https://github.com/gohugoio/hugo`
[3] `https://github.com/mattermost/mattermost-server`

in all versions of these two projects."[15]

Despite this paper's drawbacks, the similarity of their research topic to ours required further investigation. Our research suggests that using only functions with structures receivers would not provide affective TD analysis, so we wanted to ensure our assumption was correct by evaluating GodExpo. We cloned the most recent commit of GodExpo (Feb. 28, 2019)[4], Hugo v0.50, and Mattermost v5.6.0 from GitHub. Running GodExpo on those two applications at those versions returned the results in their paper. We reviewed the code then modified GodExpo to collect some additional metrics. The rest of this section discusses the findings of that investigation.

GodExpo appears to be written in Go v1.10 or earlier based on the time of the last commit and the lack of a `go.mod` file. They did not have access to Go module loaders at the time and they do not check for imports, vendored packages, nor build constraints. GodExpo will read all `*.go` files in any directory reachable from a base directory. This causes several problems. This means that tests files with any functions added for testing are also included during the GC detection, and all functions and types from any specific build file, e.g. `mattermost\vendor\golang.org\x\sys\unix\` contains twelve different build specific files `ztypes_linux_386.go`, `ztypes_linux_amd64.go`, `ztypes_linux_arm.go`, and so on. Also, those files are in the `vendor` folder, meaning they are from dependencies that Mattermost relies on and not part of Mattermost itself. The reason imports and build constraints are important is because ignoring those amplifies a bug in GodExpo discussed later on in this section.

We investigated how many types and functions there were in the two applications, and looked at how many were being used in GC analysis. We did not change the files being loaded nor any of the original code so that GodExpo continued to return the same GC estimates.

In Hugo there is a total of 504 types, 369 are types with underlying structures,

[4] `https://github.com/rafed/GodExpo/blob/1ddca9127da45addc32f73b97a5f72c28177f4a3`

66 are interfaces, and 69 are data types underlying with a non-structure type, e.g. `type SiteInfos []*SiteInfo` has the underlying type of a slice of pointers to `SiteInfo` types. Interfaces may not be used as receivers in functions, however those 69 types being ignored by GodExpo could be used as receivers and some are. Hugo has 2,809 functions containing a sum of 57,353 lines (lines of code inside of functions including comments and blank lines), 1,347 of those functions have receivers (20,738 lines), and 1,462 functions without receivers (36,615 lines). Out of the 1,347 functions with receivers, 124 functions (1,356 lines) were unassigned to a type with a structure because those functions contained receivers that were not structures, e.g. `func (decl) indexOfReplacementStart` has the receiver `type decl map[string]string`. It can be seen that 84.2% of the data types were used, and 49.3% of the functions, representing only 33.8% of the function lines, were used for the GC analysis.

In Mattermost there is a total of 7,364 types of which there are 5,611 types with an underlying structure, 384 interfaces, and 1,369 types with a non-structure underlying type. It has 29,081 functions (435,619 lines) of which there are 15,291 function with receivers (199,916 lines). 960 functions (7,484 lines) were unassigned because they had receivers types without an underlying structure. It can be seen that 80.4% of the data types were used, and 49.3% of the functions, with 44.2% of the functional lines, were used in GC analysis.

On line 171 of `main.go` in GodExpo, they use the package name and the type name in the receiver to assign a function to a type. In Go, the type name must be unique per package. However, that rule only applies when build constraints are checked, otherwise the code may define several types with the same name that should be swapped out for different builds. Also, package names are not unique. There may be two separate packages at different paths with the same name, e.g. `hugo/tpl/partials` and `hugo/tpl/templates` both have the package name `partials`. This is a bug that can cause functions to be assigned to multiple structures which means that functions are given more than one full membership and over-

reported. Any function assigned incorrectly to a type will make that type more likely to be detected as a GC.

In Hugo there are four functions assigned more than once, e.g. `Include` (45 lines) is assigned to both the `Namespace` type found in `hugo/tpl/partials` and the `Namespace` type found in `hugo/tpl/templates`. In Mattermost there are 559 functions being used in 5,764 assignments. The function `SetLen` in the package `unix` with the receiver `Iovec` is assigned 30 times.

We modified GodExpo to skip `vendor` packages and test files (`*_test.go` files), ensure build constraints, and use package paths instead of using package names. It still only assigns structure receivers and does not follow imports. After these changes, Hugo still detected seven GCs but Mattermost only detected 13 GC instead of 60. Neither has any function assigned more than once.

With these modifications Hugo now has 428 types in total, 64 interfaces, 306 types with an underlying structure, and 58 other data types. It also has 1,924 functions without receivers (31,909 lines), 3,697 functions with receivers (20,094 lines), and 120 functions with receivers that were not assigned (1,344 lines). It uses 84.1% of the data types and 59.6% of the functions with 58.8% of the function lines for GC detection.

Mattermost now has 791 types in total, 66 interfaces, 682 types with an underlying structure, and 43 other data types. It also has 5,276 functions without receivers (95,451 lines), 3,697 functions with receivers (56,357 lines), and 21 functions with receivers that were not assigned (139 lines). It uses 94.1% of the data types and 69.7% of the functions with 58.9% of the function lines for GC detection.

Most of the noise in the GC determination was coming from the `vendor` folder and assigning functions to more than one type. There is still no proof that the remaining detected GCs are being correctly determined for these applications, nor that these two applications represent the typical Go application. However, our research is attempting to use DR to

utilize all available functions regardless of how the developer of the package chose to use receivers.

Sarker, Jubaer, Shohrawardi, Das, and Siddik

Sarker, Jubaer, Shohrawardi, Das, and Siddik (2021)[16] discusses analyzing Go projects to determine five metrics for OO design and three metrics from Chidamber and Kemerer [9]. They use these metrics to identify GC and feature envy. Feature envy is detected by a method accessing the data of a class more often than it accesses the data of the class that the method is a member of. The metrics they collected are lines of code (LOC), total comments, Weighted Method Count (WMC), lack of cohesion of methods via Tight Class Cohesion (TCC), attribute hiding factor, method hiding factor, attribute inheritance factor, and method inheritance factor.

They do not describe how they are determining membership required to gather these metrics and only defined five of the metrics mathematically. The only indication of how they are determining the results they have is that they discuss GodExpo [15] in the related work. The author Siddik is one of the authors of GodExpo. Also, one of the five of the projects that they gather results for is Hugo, that was also analyzed in GodExpo, so we assume that they are using GodExpo or a very similar tool to gather several of these metrics.

To check the assumption that they are using GodExpo, we checked out the version of Hugo they used. Unfortunately they did not give a commit number nor version number. They only give the date they retrieved the source code, October 30th 2020. There were twelve commits and three pull requests merged into master on that day. A version tag was created on that day, Hugo v0.77.0, so we started with that version not knowing if they retrieved the source before or after that tag was created.

They did not specify if their LOC measurements include blank lines or comments but

they claimed that their version of Hugo had 173,958 LOC. Using cloc[5], we gathered the lines of text including blank lines from all the `*.go` files in v0.77.0, except those in the vendor folder where the project's external dependencies are stored. We found 143,341 lines. Gathering the lines of text including blank lines from all the text files, whilst still excluding the vendor folder, in v0.77.0 gives 212,756 lines. Using the same files but excluding blank lines gave 175,841, which is only 1,883 lines different from their result. That result is close enough that we can assume that is how they count their LOC metric. However, it indicates that we might have the wrong version. We got the LOC from all versions between v0.74.0 and v0.77.0, and was not able to find an exact match. The closest was v0.75.0 with 173,931 LOC that was tagged on September 14th, 2020. We decided to continue with both v0.77.0 and v0.75.0 since it was likely that they use one of the 112 commits between those two tags for their research.

They specified a single value for the "WMC" for each project. They do not specify if this is a sum and give no equation for how they calculate this value. They describe the value as the "WMC per class". We assumed this was the sum of the WMC for all classes, which is simply the sum of all the McCabe's Cyclomatic Complexity [8] (CC) in the project. For their version of Hugo they claim it got 16,268 for this value. We modified GoExpo to gather the sum of the CC for the functions selected by GoExpo as potential methods of structures, i.e. the functions with a receiver that has an underlying type of a structure, and the sum of the CC for all functions in the project. The sum of the CC for the selected functions for v0.75.0 is 7,982 and v0.77.0 is 8,067. The sum of the CC for all functions for v0.75.0 is 16,138 and v0.77.0 is 16,282. This indicates they were likely using the sum of the CC for all functions since v0.75.0 is 130 and v0.77.0 is only 14 different from their result.

Although the sum of the CC for a project included all the functions and not only those selected by GodExpo, we still felt that they are probably using something similar to

[5] `https://github.com/AlDanial/cloc`

GodExpo when determining which functions are members of which structures and ignoring all functions without a structure receiver. This is bolstered by the fact that they call GCs, *god structs*, like the GodExpo paper does. Therefore, the last value we tried to repeat was how many god structs were detected per package. For their version of Hugo, they claim it has 17 god structs. We ran an unmodified GodExpo on the Hugo v0.75.0 and v0.77.0. It returned 17 and 18 god structs respectively. The similarity suggests that our guess was likely correct, they are using GodExpo or a similar tool for detecting god structs.

Since we previously discussed how GodExpo is unproven, ignores approximately half the data and functions, and has many faults, we decided to not continue trying to determine the unspecified process they use to get the metrics they report.

CompreGo

Asad, Yasir, Shahriar, Nahar, and Tawhid (2021)[17] researched comprehensibility of a Go project. They propose the tool CompreGo[6]. A few the authors in this research are also authors in the prior two sections, GodExpo and Sarker, Jubaer, Shohrawardi, Das, and Siddik. CompreGo is similar to some TD analysis on a Go project without performing any DR meaning it cannot perform any DD analysis. They compare the comprehensibility of Go projects against Java projects using five metrics:

- *Too Long Files* (TLF) is the percentage of code stored in files that are too long. TLF is determined by taking the sum of LOC of all files over 750 lines long and dividing that by the project's total LOC. The 750 line threshold comes from research they referenced on the maintainability of large open-source projects.
- *Too Long Methods* (TLM) is the percentage of code stored in long functions. TLM is similar to TLF except the sum of LOC within functions that are over 75 lines long.

[6] `https://github.com/rafed/CompreGo`

- *Nesting Depth* (ND) is the percentage of deeply nested functions. ND is similar to TLM except the sum of LOC within functions that contain any nesting at least 5 deep.

- *Lack of Cohesive Comments* (LCC) is the percentage of non-cohesive comments on functions when compared to total number of function comments. Comment cohesiveness is determined via the Levenshtein distance between the unique words in the comment, and the function identifier separated into words assuming camel case formatting. The unique words in a comment are determined by breaking the comment into space separated parts and ignoring any parts that do not contain at least one letter. If two unique words have a Levenshtein distance of less than two, then the word is considered a match. The coherence is the total number of matches divided by the total number of unique words in the comment. A non-cohesive comment has a coherence of zero, indicating there was no comment, or above 0.5 indicating the comment is simply a restatement of the function identifier, e.g. `getWidth` with the comment "This gets the width."

- *Duplicate Comments* (DC) is the percentage of duplicate comments on functions when compared to the total number of function comments. A duplicate comment is determined if all the words in one comment has a Levenshtein distance of less than two with the corresponding words in the other comment.

Comparing the results of 50 Go and 50 Java projects, they find that "Go code has significantly higher comprehensibility than Java" for ND, LCC, and DC, "Java has significantly better comprehensibility than Go" for TLM, and no significant difference for TLF.

Their research is similar but not very useful to our research. Although they were finding metrics that could be used to rate the maintainability of a PL, their analysis does not cover

DD metrics. Similar analysis to the analysis they propose can be gotten from CodeScene's Code Health[7] and SonarQube.

God Classes

A GC, as defined by Marinescu (2004)[2], is a class that has too many responsibilities and is therefore difficult to maintain [5]. GCs have also been called God Objects, Large Class Bloaters, Monster Classes, or Junk Drawers [18]. A GC is a code smell caused by a DD anti-pattern. Three metrics are used to identify GCs: Weighted Method Count (WMC), Tight Class Cohesion (TCC), and Access to Foreign Data (ATFD).

WMC is the sum of the McCabe's Cyclomatic Complexity [8] (CC) of all methods in a class. Given a program, C is the set of all classes and $\mathcal{F}$ is the set of all methods. $B_c \subset \mathcal{F}$ is the set of all methods in class $c \in C$, defined in 2.1. Every method is a member of one and only one class as seen in 2.2 and 2.3. Given the CC for method m as CC_m, the Chidamber and Kemerer's WMC [9], for class c, WMC_c, is defined as 2.4.

$$\mathcal{B}_c = \{m | m \in \mathcal{F} \text{ and } m \text{ is a member of } c\} \tag{2.1}$$

$$\mathcal{B}_a \cap \mathcal{B}_b = \emptyset \, \forall a \in C, b \in C, a \neq b \tag{2.2}$$

$$\bigcup_{c \in C} \mathcal{B}_c = \mathcal{F} \tag{2.3}$$

$$WMC_c = \sum_{m \in \mathcal{B}_c} CC_m \tag{2.4}$$

[7] https://codescene.com/product/code-quality

TCC is the relative number of directly connected methods [10]. TCC is only calculable iff the class has at least one method. TCC is the fraction of methods that directly connect to each other in a class. A pair of methods is considered directly connected iff both methods use at least one of the same instance variables (fields) or accessor functions in a class. The methods used for cohesion calculations are methods that are visible to other classes, not including constructors and destructor functions. Given the number of methods in $\mathcal{B}_c$ defined as $\mathcal{N}_c = |\mathcal{B}_c|$ (ignoring the visibility of the methods in $\mathcal{B}_c$), the maximum possible number of direct and indirect connections in the class c, NP_c, is defined in 2.5. With NDC_c as the number of direct connections in the class, the TCC for c is defined in 2.6. Since $0 \leq NDC_c \leq NP_c$ then $TCC_c \in [0, 1]$.

$$NP_c = \frac{\mathcal{N}_c(\mathcal{N}_c - 1)}{2} \tag{2.5}$$

$$TCC_c = \frac{NDC_c}{NP_c} \tag{2.6}$$

ATFD for c is the number of classes that access the class c's fields directly or via accessor functions, $ATFD_c$. If any method in a class gets or sets data in an external (non-superclass) class, then that class accesses the other class's data.

These three metrics are gathered for all classes in the program. The metrics are then used with filters to determine which classes are likely a GC if any. The filters' thresholds may change depending on which metric the developer is choosing to emphasize and the fuzziness of the values. Marinescu [2] defines several example filters.

`TopValues` and `BottomValues` are relative semantical filters that take in the full set of a metric, the value for specific class, and the threshold. `TopValues` will be true when the given metric for the class is above the threshold percentile in the full set of that metric and `BottomValues` is for below the threshold percentile.

`HigherThan` and `LowerThan` is simply a value comparators where `HigherThan` is true when the given value is higher than the given threshold and the inverse for `LowerThan`.

Marinescu [2] provides an example GC detector as defined in 2.7. This constructs a set of GC candidate classes by checking all classes in C and adding those classes that satisfy all the filter functions. In this example detector it can be seen that only a quarter of the classes in C are selected based on the WMC for a class. The quarter with the lowest TCC are selected to further refine the set. And finally from that filtered set, only those with at least one ATFD are selected. If more than one candidate GC was selected, $|GC| > 1$, then those candidates may be sorted by those three metrics based on the developers' emphasis or the developers may choose to narrow the filters to reduce the set further.

$$GC = \left\{ c \mid c \in C, \wedge \left\{ \begin{array}{l} TopValues(WMC, WMC_c, 0.25) \\ BottomValues(TCC, TCC_c, 0.25) \\ HigherThan(ATFD_c, 1) \end{array} \right\} \right\} \tag{2.7}$$

<u>Fuzzy Sets</u>

Bezdek and Harris (1978)[19] defines the relationship between a finite data set, $X = \{x_1, x_2, \ldots x_n\}$, and a set of subclasses c, $2 \leq c \leq n$, for clustering X. They presume that each $x_i \in X$ is a representative of one or more subclasses and that this mathematical membership of the data in subclasses can be used to partition the data. Yang [20] provides a survey of fuzzy clustering algorithms and techniques, including a summary of the Bezdek and Harris paper.

$$\mathcal{P}_c = \{U \in V_{cn} \mid u_{ik} \in \{0, 1\} \, \forall i, k; \; \sum_{i=1}^{c} u_{ik} = 1 \, \forall k; \; \sum_{k=1}^{n} u_{ik} > 0 \, \forall i\} \tag{2.8}$$

Bezdek and Harris define $\mathcal{P}_c$, as shown in 2.8 as a *non-degenerate hard c-partition space* for X. It is the set of all U matrices from the set of all possible $c \times n$ matrices, V_{cn}, that

satisfy those three conditions. The first condition shows that u_{ik} is either 0 or 1, meaning that x_i is either not a member of or a full member of the subclass k respectively. The second condition shows that each x_i is a member of exactly one subclass. The third condition shows that each subclass k has at least one representative member.

$$\mathcal{P}_{co} = \{U \in V_{cn} \mid u_{ik} \in \{0,1\}\,\forall i,k;\ \sum_{i=1}^{c} u_{ik} = 1\,\forall k\} \tag{2.9}$$

If the last condition in 2.8 is relaxed to allow subclasses with no representatives, i.e. change the third condition to the tautology $\sum_{k=1}^{n} u_{ik} \geq 0\,\forall i$, then the set of all U matrices is the *degenerate hard c-partition space*, $\mathcal{P}_{co}$, as shown in 2.9. This allows subclasses with no representation, meaning that the constraint on c is no longer limited to $c \leq n$, and therefore $\mathcal{P}_c \subset \mathcal{P}_{co} \subset V_{cn}$.

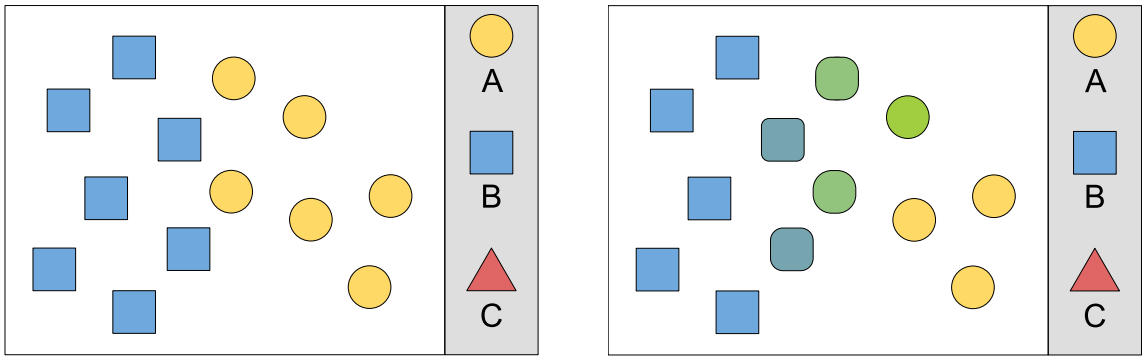
If the first condition in 2.8 is expanded to allow $u_{ik} \in [0,1]\,\forall i,k$ then the set of all U matrices is the *non-degenerate fuzzy c-partition space*, $\mathcal{P}_{fc}$. This allows u_{ik} to represent a full member ($u_{ik} = 1$), a partial member ($0 < u_{ik} < 1$), or not a member ($u_{ik} = 0$) of subclass k.

$$\mathcal{P}_{fo} = \{U \in V_{cn} \mid u_{ik} \in [0,1]\,\forall i,k;\ \sum_{i=1}^{c} u_{ik} = 1\,\forall k\} \tag{2.10}$$

The *degenerate fuzzy c-partition space*, $\mathcal{P}_{fo}$, as shown in 2.10, satisfies $\mathcal{P}_{fc} \subset \mathcal{P}_{fo} \subset V_{cn}$.

With a specific U, a relational matrix R can be generated. R is a $n \times n$ matrix where r_{ij} is the relationship between x_i and x_j. Then using composition iterations or column-row scanning, solve for the transitive closure of R, $\bar{R}$. The transitive closure is used for convex decomposition clustering of X. While iterating over R towards $\bar{R}$, a dendrogram representing the clustering can be built up.

Although this clustering can be used for our research, our research focuses on using $U \in \mathcal{P}_{fo}$ as a participation matrix and not necessarily assuming the membership between method and classes, like clustering would do.



(a) Hard (b) Fuzzy

Figure 2.1: Example Degenerate C-Partition Clusters

In figure 2.1 there are two example clusters. The points' locations are not represented in the matrices. Only the membership of a point to a classification is represented in the matrices.

In the hard c-partition cluster, 2.1a, the points are full members of A or B, and C is not represented in by any point. For our research the hard c-partition space can be used to represent an OO project. The A, B, and C are classes. A and B have the specific methods that are full methods, and C is a class with no methods.

In the fuzzy c-partition cluster, 2.1a, some points may be full members of A or B, some are partial members of both A and B, and C is not represented. The fuzzy c-partition space can be used to represent an PL project. The A, B, and C are objects and the functions may be full or partial members with those objects. An object like C does not participate in any function in a way to be represented by any function.

Design Recovery and Migration

DR techniques were developed to help with migration from non-object oriented languages, such as PL, into OO or mixed paradigm languages, or to partition a program to increase maintainability.

Biggerstaff (1989)[21] states "Design recovery recreates design abstraction from a combination of code, existing design documentation (if available), personal experience, and general knowledge about problem and application domains." Chikofsky and Cross (1990)[22] refines DR as "... a subset of reverse engineering in which domain knowledge, external information, and deduction or fuzzy reasoning are added to the observations of the subject system to identify meaningful higher level abstractions beyond those obtained directly by examining the system itself."

The overall concept is the same across all of the DR papers; estimate how closely a function is related to a set of data, then prune those estimates to determine the functions that are likely members of a set of data. The papers discuss having human interactions or additional design information provided to facilitate the pruning process.

Belady and Evanglisti

Belady and Evanglisti (1981)[14] define a basic process for partitioning program modules and data structures into maintainable sets. The partitioning is based on the interconnections in the program as created by calls and references. They never specifically specified the language that the project, Virtual Telecommunications Access Method (VTAM), was written in, nor discuss classes. However, the concept of grouping similar modules together for maintainability is fundamentally the same as grouping functions with sets of data to create classes.

To perform the clustering they created a graph with a node for each module. If the modules are interconnected then an edge is added between the nodes for those modules. The graph is converted into an MxM matrix where M is the number of modules and with a one in the matrix indicating a connection between two modules. They then picked N eigenvectors to cluster the modules into N-space.

They define a complexity heuristic as the number of edges between clusters multiplied

by the number of nodes in a cluster. The complexity of the whole system is normalized with the total possible connections between clusters, thus making this metric similar to Bieman and Kang's class cohesion [10] metric. They show that N is typically five for VTAM based on minimizing that complexity heuristic. The goal is to create module clusters that maximize the number of connections internal to the cluster whilst minimizing the number of connections between two clusters.

Hutchens and Basili

Hutchens and Basili (1985)[23] extend Belady and Evangelisti's work and others by adding a *data binding* metric to better define the interconnectedness of modules. They used data bindings to measure the interfaces and create a hierarchical clustering. In their paper, interfaces are similar to how Params (1972)[12] described them, as the connection between modules, and not like an OO interface described by Liskov and Wing (1994)[13]. They described this work for the programming language Ada. Ada, at the time of their paper, had not yet received the update to support OO paradigm concepts and still followed PL paradigms with modules containing data and functions.

The data binding is defined to have several levels starting with *potential data binding*. Potential data binding, (p, x, q) is defined as two different functions, p and q that both are in scope of the same variable, x. Neither function has to actually access the data to be considered since the possibility still exists that the functions could be modified to communicate via that variable without the variable's scope being modified. The potential data binding becomes *used data binding* when both p and q use x for either reference or assignment. Used data bindings match the interconnectedness measurements used by Belady and Evanglisti [14]. Used data binding becomes *actual data binding* when p assigns a value to x and q references x. The order of execution for p and q are not considered. In the cases where the order of execution can be determined from set of rules for the static code analysis,

the actual data binding is promoted to *control flow data binding*. Control flow data binding indicates a likelihood that the functions are communicating via x despite the static code analysis being unable to determine if p and q will both execute in all cases on each run.

The data bindings are used to construct a dissimilarity matrix given a function of dissimilarity. From the dissimilarity matrix the dendrogram of module clusters can be constructed. Hutchens and Basili provide a few example functions to help explain how to construct the dissimilarity matrix, such as the following.

Given B as the set of control flow data bindings, and F is the set of all functions in the program, then sum_i is the number of data bindings containing function i in the triplet, and sum_{ij} is the number of data bindings containing both i and j, i.e.(i, x, j) or (j, x, i). The dissimilarity matrix D is then defined in 2.11.

$$D = \begin{cases} D_{ij} = 0 & i = j \\ D_{ij} = \frac{sum_i + sum_j - 2sum_{ij}}{sum_i + sum_j - sum_{ij}} & i \neq j \end{cases} \quad \forall i, j \in F \tag{2.11}$$

Their paper discusses some changes to the dissimilarity matrix to handle different issues that may occur while constructing the dendrogram. They discuss a *planetary* system where the modules form a star pattern around a central cluster. The central cluster tends to be larger but in some systems it may be very small making the system resemble a ring. A *black hole* system where the clustering finds one subset of modules that absorb the rest of the system. Another is a *gas cloud* where the modules tend to not cluster well because of a large degree of coupling between the modules.

Hutchens and Basili shows concern about the space and complexity needed to calculate the different levels of data bindings. When their paper was written, attempting to use the highest level for best clustering could be prohibitively large or take too long for large systems. The levels are to allow the developer to decide what is important to them and what is possible for their hardware when determining the custom dissimilarity function.

Biggerstaff

Biggerstaff (1989)[21] describes a human assisted DR using the source code, any design documentation, personal experience, and general knowledge about a program. He discusses the complications of DR for design reuse, reverse engineering, and pattern identification. His work focuses on the PL C. He provides a short discussion about the changes his process would require for OO. The process he defines requires a developer to work with the results generated by the analysis in order to interpret and refine useful information. The analysis system examines the importance of informal information, such as the names used for classes, methods, and data as well as developers comments. He defines the process of DR in three steps to help a developer understand a program.

The first step is abstracting modules and features from the program. Biggerstaff does not explicitly define how to abstract modules from the program nor reference Hutchens and Basili, however it seems clear that part of step one could be filled by an algorithm similar to Hutchens and Basili's clustering. The clustering Biggerstaff describes keeps references back to the static code structures so that other semantic information, such as code comments or an issue-based information system (IBIS), can be easily referenced later by the developer. Modern day IBIS could be considered to be JIRA[8] or the issue tracker in GitHub[9] and programmatically accessible using a tool similar to SZZ [24].

The second step is for the developer to use the clustering from the first step to create or update a reuse and recovery library. The reuse and recovery library is a design diagram that can be queried to help in the next step. Biggerstaff doesn't go into details about how this functions beyond some generalizations while discussing a prototype implementation.

The third step is to apply the results of the DR by refactoring the code around the most distinct candidates in need of work. The developer performs the refactoring using the

[8] `https://www.atlassian.com/software/jira`
[9] `https://github.com/features/issues`

reuse and recovery library diagrams and the clustering of modules. During refactoring the developer could alter the design abstraction information to match with the code changes or rerun the DR to identify new candidates for refactoring.

The last two steps are not important to our research since they require significant amount of human interactions or are replaced by analytics from TD analysis. However the description of cyclical refactoring resembles modern application refactoring and CI/CD by practitioners using TD analysis tools.

Liu and Wilde

Liu and Wilde (1990)[25] propose a system similar to Biggerstaff, however, they focus more on the analytics that can be programmatically derived from static source code. They expand on Hutchens and Basili and other similar works to group functions and data together into possible classes. Unlike prior research that discuss their approaches in terms of modules and components, Liu and Wilde are specifically looking at objects while equating them to OO classes. They also discuss that their DR could be used to assist rewriting a PL program in an OO language.

Liu and Wilde define a candidate object, (F, T, D), where F is the set of functions, T is the set of types, and D is the set of data items. The types are the structured data being acted on. The data items are actual instances of object classes. If the candidate object is used as a class, then the set of functions will be members of that class. As the candidates are resolved, the sets should contain unique functions, types, and data items that do not overlap with other candidate sets. Any set in the candidate object may also be empty.

They discuss that during the DR a useful partial classification of the candidates is meaningful but requires human intervention to carefully remove any meaningless or coincidental relationships. They provide two procedures for finding objects that they found useful.

The first object finding procedure is based on global data and clusters the functions that uses (reads from or writes to) that data. This is similar to Hutchens and Basili's metric based on shared data between functions.

Given all the functions in the whole program, $\mathcal{F}$, and all the data in the whole program, $\mathcal{X}$, then the set of functions that access some data $x \in \mathcal{X}$ is P_x defined in 2.12. Construct a graph, $G = (V, E)$, where each node in V is a global variable with at least two functions using it, as defined in 2.13, and each edge in E is the overlapping set of functions for data as defined in 2.14. For any strongly connected component (SCC), $C_i = (V_i, E_i)$ in G, a candidate object is defined as 2.15.

$$P_x = \{m \mid m \in \mathcal{F}, m \text{ uses } x\} \tag{2.12}$$

$$V = \left\{P_x \;\middle|\; |P_x| \geq 2 \; \forall x \in \mathcal{X}\right\} \tag{2.13}$$

$$E = \{P_i P_j \mid \; i \in \mathcal{X}, j \in \mathcal{X}, i \neq j, P_i \cap P_j \neq \emptyset\} \tag{2.14}$$

$$(F, T, D)_i^1 = \left(\bigcup_{P_x \in V_i} P_x, \; \emptyset, \; \bigcup_{P_x \in V_i} x \right) \tag{2.15}$$

The second object finding procedure defines a binary matrix describing type and method interactions while taking into account subtypes. First define the topological order of types where $x \ll y$ denotes that x is a sub-type of y as defined by Liskov and Wing [13]. Create a relationship matrix, R, where the rows are functions and the columns are formal parameters and result types. Initialize each row, $f \in \mathcal{F}$, with a one for any sub-type of a parameter or result types and a zero otherwise. Then refine that row by replacing any one with a zero iff the super-type of that type has a one in that row. Collect functions into maximal groups,

max_i, based on sharing data, t, i.e. $R_{f_1,t} = R_{f_2,t} = 1$. Each maximal group is a candidate object as defined in 2.16.

$$(F, T, D)_i^2 = \Big(\{f \mid f \in max_i\},\ \{t \mid R_{f,t} = 1 \text{ for any } f \in max_i\},\ \emptyset\Big) \tag{2.16}$$

Liu and Wilde discuss how a developer could leverage multiple object finding procedures by selecting candidate objects then reducing the input into the next set of object finding procedure by removing the previously selected candidates' functions, types, and data.

Livadas and Johnson

Livadas and Johnson (1994)[26] extends Liu and Wilde object finding procedure and adds a third metric based on the receiver parameter types in a method.

They rename the first object finding procedure from Liu and Wilde as *global-based object identification* (GBOI). They discuss an improvement to this algorithm by performing flow-sensitive analysis. This allows for accessors called by another function and functions with access to global data that passes that data into another function, to be taken into account when clustering, e.g. identifying a function that only returns a value, similar to accessing that value directly.

They rename the second object finding procedure from Liu and Wilde as *type-based object identification* (TBOI). They discuss how Liu and Wilde's algorithm used parameter types, tp, and return types, tr, but ignore directly using global types, tg. They update the algorithm to allow any combination of those three sets of types when clustering, e.g. $tp \cup tr$ matches Liu and Wilde's algorithm, and $tp \cup tr \cup tg$ uses all three sets.

They introduce a third object finding procedure called *receiver-based object identification* (RBOI). They define a receiver as the "type of a parameter that is modified in at least one execution path" of a method. This definition is different from modern receiver definitions, such as `this` in some OO or the optional receiver parameter in Go. Livadas and

Johnson's receivers are the set of all parameter types, rp, and global types, rg, that could be mutated by the function. This object finding procedure will not include non-mutating functions, like an accessor function. The rest of this object finding procedure is similar to TBOI.

Livadas and Johnson describe how to aggregate candidate object sets from different object finding procedures based on developers' needs. As candidates are selected they can be removed from candidate objects.

Sneed and Nyáry

Sneed and Nyáry (1995)[27] is an example of several domain specific software conversion papers. They specify a migration from mainframe source written in COBOL being moved to networked work stations written in Object-COBOL or another OO. For example, they discuss breaking up the source code based on the MVC (model, view, controller) implementation for a database with a visual output.

These domain specific software conversion papers relate to DR and PL to OO migrations, but are not as beneficial in our research. Most other similar domain specific papers have been omitted, except for a few that provide relevant information to our research.

Cimitile, De Lucia, Di Lucca, and Fasolino

Cimitile, De Lucia, Di Lucca, and Fasolino (1999)[28] is similar to Sneed and Nyáry where they discuss how to migrate a COBOL program. However, they used Livadas and Johnson, and other prior works to create a bipartite graph. The graph defines the weighted relation between *program units* (similar to functions) and *data stores* (similar to structures).

Given the set of program units is PP and the set of data stores is DD then $n = |PP|$, $m = |DD|$, and the edges are assigned a weight, $0 \leq a_{ij}\ \forall i \in PP, j \in DD$, via an association method. For example, a_{ij} could be assigned to the number of times the program unit i accesses the data store j.

Using the bipartite graph, they create a binary matrix, $X_{n\times m}$, with the $x_{ij} = \{0, 1\}\ \forall i \in PP, j \in DD$. If $a_{ij} = 0$ then $x_{ij} = 0$, meaning that if $x_{ij} = 1$ then $a_{ij} > 0$. However, they add the constraint $\sum_{j\in DD} x_{ij} = 1\ \forall i \in PP$, meaning that a program unit may only be associated with one data store. Although they don't say it in the paper, the definition of X matches the definition of a degenerate hard c-partition space, P_{co}, as seen in 2.9. In terms of our research, X is a membership matrix, discussed more in the section Membership and Participation.

$$WMC_j = \sum_{i\in PP} c_i x_{ij} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{|PP|} \end{bmatrix} X \tag{2.17}$$

They show that X can be used to calculate Chidamber and Kemerer's WMC [9], as shown in 2.17, with c_i being the Cyclomatic Complexity (CC) for the program unit i. This is a matrix version of 2.4.

They describe different strategies for producing a_{ij} values and an objective function, $f(X)$, for determining the data store that a program unit is associated with, i.e. which x_{ij} are set to one. Any $X \in P_{co}$ should minimize (or maximize) $f(X)$.

For example, minimizing $f(X) = max_{j\in DD}(WMC_j) - min_{j\in DD}(WMCj)$ will distribute the CC more evenly across data stores. This distribution will minimize Potpourri Modules [11]. However, a uniform distribution of complexity may not be meaningful since it is not taking into account the number of times a data store is accessed.

They propose maximizing $g(x) = \sum_{i\in PP}\sum_{j\in DD} x_{ij}a_{ij}$. This simplifies to $x_{ij} = 1$ iff $a_{ij} = max_{k\in DD}(a_{ik})$. If a program unit can associate to more than one data store equally, then the most predominantly associated data store is used based on a heuristic passing a threshold, as they describe in the paper.

They describe how to decompose the program into smaller modules once an X has been

selected. The clustering of data stores with associated program units is determined using a call graph. This is specific to migrating the program, but is not related to our research.

Design Transformations

Pidaparthi and Cysewski (1997)[29], Pidaparthi, Zedan, and Luker (1997-1998)[30, 31], González, Czarnecki, and Pidaparthi (1998)[32], and Pidaparthi, Luker, and Zedan (1999)[33], propose design transformations and translations to convert a PL program into an OO program while keeping the overall design similar. They specify that their method is to assist in transforming PL programs into OO programs and not an attempt to create a translation into the target language. They define translation as converting "...a program from one language to another while preserving the software architecture of the system."[30] They define design transformation as a conversion "...in which the organisation of components of the system is modified, based on inter component relationships."[30] These two processes are orthogonal such that they can be performed in any order during migration.

The different papers discuss different aspects of the transformations while the overall method of performing the transformations are the same. Their translation methods are based on Livadas and Johnson, Liu and Wilde, and other works. The object identification portion is similar to the prior papers, however, once objects are identified a selected set of transformations are applied to produce a more maintainable design. The example they use in their paper is a set of transformation that will break up a GC into smaller objects automatically. Although the specifics of the transformations are not important to our research, the fact that the DR has already been used in association with identifying TD adds credence to our hypothesis. The design translation, mostly defined in prior papers, is more useful for TD and our research because it attempts to preserve the original architecture.

Zou and Kontogiannis

Zou and Kontogiannis (2001)[34] research is a domain specific study of migrating PL to OO server frameworks. They take the Abstract Syntax Tree (AST) from a PL source code and convert it into an XML file. When converting, they use a domain model that defines the Document Type Definition (DTD) to write the AST as XML. The domain models are specific to the languages being converted. The resulting DTD and AST is used to abstract objects to be used during transformation of the source code into the OO equivalent.

They extract the objects from the AST with "a series of iterative analysis steps."[34] They identify functions that are candidates to be attached to extracted objects. They use global data types, global variables, parameters lists, and return types of a function to determine which object the function may be associated with. They then use quality measurements to refine the candidate functions, such as by minimizing class coupling and maximizing class cohesion. The result of the candidate selection includes an *evidence model* used by the developer performing the migration to resolve conflicts.

They want to create code that leverages more of the OO paradigms, such as inheritance and polymorphism, than simply grouping functions and data into classes. They apply additional analysis to the candidate objects to look for data field cloning among classes, function code cloning, type casting, and other indicators that suggest inheritance. The evidence model contains metrics to help the developer performing the migration decide when long functions should be broken up into smaller functions across several objects. The metrics also look at the number of casts and type switching statements to indicate when an interface should be created and data accessors should be added to leverage polymorphism in a function.

Based off of a few empirical tests, the additional data provided by the AST, DTD, and evidence model, improved the readability, maintainability, and ease of migrating PL server projects in C to OO in C++.

Their research is relevant to our research since they take a similar approach to model

abstraction from source code that can be used to perform DR. Our research uses the AST from PL and OO projects to construct a JSON file similar to their XML file describing the data types and functions in the project. Our Go abstractor determines which data types duck-type to different interfaces and constructs an interface hierarchy. The JSON file is then passed into a tool to perform DR for the project thus mirroring their process until they hand off the file to developers where as we instead perform TD analysis. Our research does not use a DTD, instead we use a language agnostic information.

Hossain and Zulkarnine

Hossain and Zulkarnine (2011)[35] discuss five metrics for analyzing design structures. Their research focuses on domain specific study of large scientific software for simulations and engineering applications. "A distinguishing feature of the designers of such software applications is that they are highly trained scientists with little or no formal background in modern software engineering practices."[35] Since their research is more about design analysis of a specific domain, it is not very useful for our own research. However, it is useful to help contextualize the following research papers.

The metrics Hossain and Zulkarnine discuss are conventional graph theory metrics being gathered from a *call graph*: *characteristic path length*, *clustering coefficient*, *nodal degree*, *strongly connected components*, and *propagation cost*. The call graph is created from a project's static code and can be represented by a dependency structure matrix (DSM). Given the set of all functions in the project, $\mathcal{F}$, then the DSM is a $|\mathcal{F}| \times |\mathcal{F}|$ matrix and all the vertices in the call graph are simply $\mathcal{F}$. An edge between function $i \in \mathcal{F}$ and $j \in \mathcal{F}$ represents function i invoking function j in at least one control flow path through function i's body.

The projects they use in their research create a mostly triangular DSM attributed to the projects having mostly acyclic function calls without much recursion. They explain that

based off of prior work it is better to treat the edges as undirected for characteristic path length and clustering coefficient. They justify this by citing prior work that look into flow of data and how the flow of data in a function call is bidirectional via the functions' parameters and results.

Siddik, Gias, and Khaled

Siddik, Gias, and Khaled (2013)[36] discusses migrating PL programs to OO to provide better modularity, manageability, and extendibility. They want to determine how to perform an automatic migration (transformation) into a more maintainable OO design. Their research topic pertains to our research and their research leverages Hossain and Zulkarnine's research.

They state "The hypothesis that the methods of the same *class* will call each other more frequently than the number of calls between methods of different classes has been used on the call graph."[36] They based this idea on encapsulation as discussed in Chidamber and Kemerer[9] and Bieman and Kang [10]. Code with less coupling between objects is easier to maintain. They cluster the call graph with the criteria to maximize the number of intra-cluster edges, i.e. edges between clusters, and minimize the number of inter-cluster edges, i.e. edges within a cluster. "An optimal clustering of the call graph produces [a] number of non-overlapping vertex clusters that represent groups of closely related functions in the [PL] code."[36] Then, according to them, each cluster indicates a class or interface. They discuss how the calls internal to a cluster will remain mostly unchanged during migration but the calls between clusters would need to be rewritten to create proper interfaces as described by Parnas [12].

They try five Monte Carlo schemes for clustering with random seeds and compared them against three greedy based heuristics. They use characteristic path length and clustering coefficient to measure the quality of the clustering. They run the clustering on three projects:

two generated from the scientific software in Hossain and Zulkarnine, and one synthetically created. They do not describe the difference between generated code and synthetically created code. None of the source code for the three projects are provided. The different clustering algorithms found one to eight classes, two to fourteen classes, and one to three classes respectively in the three projects. They do not discuss if any of the clusters of functions made logical sense for the application design nor if the code would actually be easier to maintain.

Although it is an interesting idea to cluster functions by function calls and it might produce lower coupling, to us it does not seem likely that the functions would have a logical grouping around data. This would likely complicate the understandability of the code.

For example, imagine a point class with two hidden fields, x and y. The class has accessors to get and set x and y individually. Those accessors never call each other but shared data between the setter and getter means those functions can communicate between that data. Belady and Evanglisti [14] discussed this when using function calls and data references for analysis. Clustering using only functions calls could put the setter and getter for the same value of the point class in separate clusters. This means the values could not be hidden so that they can be accessible from different classes, thus not following encapsulation and data hiding as described by Parnas [12]. It could make the code hard to maintain because functions from one class would access foreign data from another class, such as would happen when the setter function is clustered into a different class than the associated getter. Ignoring the data could cause the class cohesion to decrease.

A class needs to make a logical grouping of data and functions that is understandable by developers and may represent a real-world entity or concept that cannot be described by code alone. When maintaining the code and adding new features, functions need to be grouped with the logical class so that it can be easily discovered by developers other than the original writer. If the functions are instead grouped closer to the consuming class because

that function is only called by the consumer, then maintainers may not see the function when creating a new consumer and create a new copy of the function, thus duplicating effort and code. A logical class that is easy to maintain, could become very hard to maintain if spread across several other classes, because a change to one class may require other classes to be changed to make the code function or prevent bugs. Therefore this clustering could increase the probability of bugs and increase the time to maintain the code over the code being logically grouped [12].

Several design patterns may also suffer by clustering functions based only on calls such as a proxy, a facade, or a bridge pattern [37] since those tend not to have as many internal calls. Maximizing the internal calls when clustering could cause the patterns to be broken up across several other classes.

Without providing any evidence that the clusters properly represent the classes that are intended by the developer and source code, Siddik, Gias, and Khaled only provide a mechanism to increase or decrease some metrics. Since our research is more about representing the existing designs as they are without transforming the design, the clustering of functions alone is not very helpful to our research.

It should be noted that one of the authors in this and most of the following functional clustering papers, Siddik, later worked on the GodExpo paper discussed in the GodExpo subsection and the Go analysis paper discussed in the Sarker, Jubaer, Shohrawardi, Das, and Siddik subsection.

Selim, Siddik, Gias, Abdullah-Al-Wadud, and Shah

Selim, Siddik, Gias, Abdullah-Al-Wadud, and Shah (2014)[38] extends the previous paper [36] using a genetic algorithm to cluster the function calls instead of a Monte Carlo algorithm. They continued to use characteristic path length and clustering coefficient as quality measurements to maximize intra-cluster function calls and minimize inter-cluster

calls. They compared the genetic algorithm it against one of the greedy and one of the Monte Carlo clustering algorithms. They used three similar programs to the Monte Carlo paper. Two of the programs seem identical based on reported metrics. However the synthetic program seems to be different from the synthetic program used in the Monte Carlo paper, since it got different results for the Monte Carlo and greedy clustering algorithms than the synthetic program used in the prior paper. The genetic algorithm created the same number of clusters as the greedy algorithm but achieved better clustering than the Monte Carlo and the greedy clustering according to the quality measurements. They do not discuss the maintainability nor understandability of the created clusters when viewed as classes.

Siddik, Gias, Selim, Khaled, and Sakib

Siddik, Gias, Selim, Khaled, and Sakib (2014)[39] extend the previous paper [38] using two local search heuristics when clustering instead of a generic algorithm. They use the same three projects as the genetic paper and the same quality measurements. They compared the two local search heuristics against the Monte Carlo, greedy, and genetic results from the genetic paper. They demonstrated that both local search heuristic clustering algorithms are better than the genetic algorithm according to the quality measurements. They do not discuss the resulting classes created from the clusters.

Siddik, Selim, Gias, and Khaled

Siddik, Selim, Gias, and Khaled (2016)[40] extend the previous paper [39] using variable neighborhood search based approach to clustering function calls instead of local search heuristics. This is the first paper in these function clustering papers to reference DR research, specifically they discuss Pidaparthi and Cysewski [29], Liu and Wilde [25], Sneed and Nyáry [27], and a few other domain specific migrations not related to our research. They dismiss the DR research with claims such as “prone to create large classes”, “may generate flawed object design”, and “is not fully automated” [40] without providing any evidence for those claims.

They use the same programs and quality measurements as the genetic clustering paper [38]. They compare the prior papers' results against two new variable neighborhood search algorithms and demonstrated that variable neighborhood search could be better according to the quality measurements. They do not discuss the resulting classes created from the clusters.

Islam, Toma, Selim, Gias, and Khaled

Islam, Toma, Selim, Gias, and Khaled (2016)[41] extends the prior papers for clustering function calls to incorporate prior research into DR, including ones not discussed in the variable neighborhood search paper [40]. They point out some problems with clustering by function calls only.

They discuss creating a *Weighted Data Call Graph* ($WDCG$). Similar to the call graphs in prior papers, the $WDCG$ has a vertex for each function, however, they also include a vertex for every global variable and constant. They call each function vertex a *function node* and each variable or constant vertex a *data node*. They discuss how functions and data from the system or external libraries can be ignored since those are not candidates entities in the migration. If F is the set of functions and D is the set of data in the source code that are candidate entities in the migration, then $N = F \cup D$ is the set of all nodes.

There are four types of directed edges between the nodes and each edge can be weighted. The weighting is dependent on the type of edge. A *Call Edge* is an edge added between two function nodes based on whether one function calls the other, the same as the call graph's edges. Every node also has a *Self Edge* that connects that node to itself. The self edges are used to store the weighting for that node itself. Similar to other DR papers, there are *Read Edges* and *Write Edges* as well. A read or write edge connects a function node to a data node depending on whether the data is read or written to respectively within the function. Reading a data node includes any data passed as an argument to the function. Writing a

data node includes any writes to the value of a pointer or reference argument to the function. Two nodes may have more than one edge of different types between them. The set of these four edge types is K.

They say that the weights for call, read, and write edges are constants that can be adjusted. The self edges weighting is always set to one. For the application they discuss in the paper they assign the weights of call edges to 0.1, read edges to 0.2, and write 0.4.

The *entity map* of the functions, variables, and constants can be determined from the $WDCG$ as shown in 2.18. The entity map for the entity $i \in N$, $\mathcal{E}_i$, contains a tuple for all edges connected to i. Each tuple contains the entity, $j \in N$, that the edge connects to and the weight, w_{ijk}, of the edge of type $k \in K$ connecting i to j.

Since there could be several types of edges connecting i to j, they propose a *merged entity map*, $\mathcal{M}_i$, as shown in 2.19, that replaces all entries to j with one entry to j with the sum of the weights of all edge types that connect those two nodes.

$$\mathcal{E}_i = \{(j, w_{ijk}) \mid \ (i, j, w_{ijk}) \in WDCG \ \forall j \in N, k \in K\} \tag{2.18}$$

$$\mathcal{M}_i = \left\{ (j, \sum_{k \in K} w_{ijk}) \mid \ (j, w_{ijk}) \in \mathcal{E}_i \ \forall j \in N, k \in K \right\} \tag{2.19}$$

They define the following operands that can be applied to merged entity maps: The *weight sum*, $S(\mathcal{M}_i)$, gets the sum of all the weights in $\mathcal{M}_i$. The *element set*, $E(\mathcal{M}_i)$, gets the set of all entities in $\mathcal{M}_i$. The *weighted intersection*, $\mathcal{M}_i \bigcap_w \mathcal{M}_j$, gets a set of all the tuples from both entity maps that has an entity in $E(\mathcal{M}_i) \cap E(\mathcal{M}_j)$. The *weighted union*, $\mathcal{M}_i \bigcup_w \mathcal{M}_j$, gets a set of all the tuples from both entity maps.

Using those operands, they define a *weighted similarity coefficient* (J_w) between two merged entity maps, as shown in 2.20, based on the Jaccard Similarity between two finite sets.

$$J_w\left(\mathcal{M}_i, \mathcal{M}_j\right) = \frac{S\left(\mathcal{M}_i \bigcap_w \mathcal{M}_j\right)}{S\left(\mathcal{M}_i \bigcup_w \mathcal{M}_j\right)} \in [0, 1] \tag{2.20}$$

Using the J_w between all merged entity sets in $WDCG$, they define a *weighted distance matrix* (WDM), as shown in 2.21. The distance between two merged entity sets is one minus the J_w of those two sets such that zero means the sets are the same and one means the sets do not have any entities in common. The WDM is a symmetric square $|N| \times |N|$ matrix with zeros along the diagonal.

$$WDM_{i,j} = 1 - J_w\left(\mathcal{M}_i, \mathcal{M}_j\right) \tag{2.21}$$

To cluster the entities into classes they perform agglomerative hierarchical clustering using the WDM for a $WDCG$. They implement the clustering to start with $|N|$ clusters each containing one entity. Each iteration will select the closest pair of clusters to join them together. The joined clusters form a hierarchy that replaces those two clusters with the one joined cluster in the next iteration. When done there will be $|N|$ hierarchies in one cluster.

The candidate objects are represented somewhere within those hierarchies. They measure the quality of a candidate object using the metrics high cohesion, low coupling, and high granularity. These metrics are used in the same way the Monte Carlo paper [36] used them, by maximizing the intra-cluster edges and minimizing the inter-cluster edges. Instead of performing clustering on all the vertices, the clusters already in the hierarchies are used. Moving from the leaves up the hierarchies, take a measurement of the two clusters in a join and compare against the cluster created by the join. This may not be able to produces as high of quality, according to the quality measurement, results as prior papers because of the additional constraints caused by taking data into consideration.

To evaluate the results, they used five programs and four experienced developers. The developers manually clustered the programs based on their understanding of the code. They

use the Jaccard Similarity Coefficient to determine the similarity of their results and the results of prior papers against the version of the programs written by the developers. Their results included variations of the weights. They determine that the best weights are 0.1 for calls, 0.4 for reads, and 0.5 for writes. Their results show a significant improvement in similarity to the developer written code than the greedy, Monte Carlo, genetic, and nearest neighbor approaches from prior papers.

Samsuddoha, Siddik, Selim, and Khaled

Samsuddoha, Siddik, Selim, and Khaled (2017)[42] perform a repeat empirical evaluation of the results found by the prior agglomerative hierarchical clustering paper [41]. They chose six projects: "Phone", "Snake Game", "Department Store", "Bank Manag. System", "Medical Store", and "Silver Searcher". The projects range from 15 to 71 functions and 550 to 3,500 LOC. They do not provide the source code for any of those projects.

They have a software engineer cluster the code into classes to use as the expected results to compare against. The comparison uses the Jaccard Similarity Coefficient to determine which algorithm produce the most similar result. They do not specify the weights and clustering limits they use for agglomerative hierarchical clustering, but the results they got seem high to us when compared to the number of functions in the project.

They found the genetic algorithm [38] performed the best overall. The agglomerative hierarchical clustering performed better for smaller projects. They check the algorithms from the local search [39] and variable neighborhood search [40] papers and determine that those have worse results than the other two.

Membership and Participation

Our prior works include Nelson and Izurieta (2021)[7] where we identify the gap in TD analysis for PL, specifically Go, and Nelson, Izurieta, and Reimanis (2023)[43] where we

provide the conceptual foundation for a participation matrix. The gap in TD analysis for PL is discussed in the Introduction. This section will focus on our discussion of the membership matrix and the proposal for a participation matrix.

In our prior work, we discuss how a membership matrix is used in current TD analysis techniques designed for OO, such as Marinescu's GC detection [2], seen in the God Classes section. The three metrics used to identify GCs are WMC, TCC, and ATFD. All three of those depend on knowing the methods' memberships to classes. In the subsection Cimitile, De Lucia, Di Lucca, and Fasolino, WMC metric is shown in a matrix form. We define the membership matrix, M, more formally to help our discussion of the participation matrix.

M will have all the methods in the whole program ($\mathcal{F}$) as the columns and all the classes in the whole program ($\mathcal{C}$) as the rows, as seen in 2.22. M is populated by writing a one between all methods and classes that have a defined membership, otherwise zero, as seen in 2.23. Since a method may be a member of only one class, the sum of each column will be one, as seen in 2.24. The rule in 2.24 is the same as the constraint in 2.9.

$$M_{|\mathcal{C}|\times|\mathcal{F}|} = \begin{bmatrix} M_{1,1} & M_{1,2} & \cdots & M_{1,|\mathcal{F}|} \\ M_{2,1} & M_{2,2} & \cdots & M_{2,|\mathcal{F}|} \\ \vdots & \vdots & \ddots & \vdots \\ M_{|\mathcal{C}|,1} & M_{|\mathcal{C}|,2} & \cdots & M_{|\mathcal{C}|,|\mathcal{F}|} \end{bmatrix} \tag{2.22}$$

$$M_{c,m} = \begin{cases} 1 & \text{if } m \text{ is a member of } c \\ 0 & \text{otherwise} \end{cases} \tag{2.23}$$

$$\sum_{c\in\mathcal{C}} M_{c,m} = 1 \quad \forall\, m \in \mathcal{F} \tag{2.24}$$

It can be seen that the M is a degenerate hard c-partition space matrix, $M \in \mathcal{P}_{co}$, as defined in 2.9 as part of the Fuzzy Sets section. In this case the data being partitioned into clustering subclassifications is methods being clustered with classes. Some OO, such as Java,

will not allow global methods to be defined outside of any class. In those cases, M is in a non-degenerate hard c-partition space, $M \in \mathcal{P}_c$.

We propose that we could expand the membership matrix space to a degenerate fuzzy c-partition space as defined in 2.10. We believe this would allow an estimate of membership to be used in TD analysis. Instead of memberships between methods and classes, the estimate of membership represents how objects' participate in a function's purpose. Therefore, we propose the *participation matrix*, $P \in \mathcal{P}_{fo}$.

P has the same rows and columns as a M, as seen in 2.25. The main difference is that M has only zeros and ones (i.e. hard) where as P has a value in the range of zero to one inclusively (i.e. fuzzy), as seen in 2.26. The fuzzy estimate set of objects for a function, i.e. a column, must follow a rule similar to 2.24 and the constraint on 2.10, as seen in 2.27.

$$P_{|\mathcal{C}|\times|\mathcal{F}|} = \begin{bmatrix} P_{1,1} & P_{1,2} & \cdots & P_{1,|\mathcal{F}|} \\ P_{2,1} & P_{2,2} & \cdots & P_{2,|\mathcal{F}|} \\ \vdots & \vdots & \ddots & \vdots \\ P_{|\mathcal{C}|,1} & P_{|\mathcal{C}|,2} & \cdots & P_{|\mathcal{C}|,|\mathcal{F}|} \end{bmatrix} \tag{2.25}$$

$$P_{c,m} \in [0,1] \subset \mathbb{R} \tag{2.26}$$

$$\sum_{c\in\mathcal{C}} P_{c,m} = 1 \quad \forall\, m \in \mathcal{F} \tag{2.27}$$

To use either the M or P for TD is trivial. We demonstrate by showing WMC being calculated with a matrix similar to 2.17, instead of with the set of membership methods as shown in 2.4. Given a vector of the Cyclomatic Complexity [8] (CC) for each function, CV, as seen in 2.28, the WMC for the whole project is calculated by multiplying M by CV. If P is multiplied by CV, then the result is the estimated WMC, WMC^E.

$$CV_{|\mathcal{F}|\times 1} = \begin{bmatrix} CC_1 \\ CC_2 \\ \vdots \\ CC_{|\mathcal{F}|} \end{bmatrix} \tag{2.28}$$

$$WMC_{|\mathcal{C}|\times 1} = M\left(CV\right) = \begin{bmatrix} WMC_1 \\ WMC_2 \\ \vdots \\ WMC_{|\mathcal{C}|} \end{bmatrix} \tag{2.29}$$

$$WMC^E_{|\mathcal{C}|\times 1} = P\left(CV\right) = \begin{bmatrix} WMC^E_1 \\ WMC^E_2 \\ \vdots \\ WMC^E_{|\mathcal{C}|} \end{bmatrix} \tag{2.30}$$

The rules that columns must sum to one, i.e. 2.24 and 2.27, is to ensure that a metric is not over-reported nor under-reported. For example, every project has a total CC, and regardless of how the individual functions are clustered with objects for WMC, that total must remain the same, as seen in 2.31.

$$CC_{total} = \sum_{m\in\mathcal{F}} CC_m = \sum_{c\in\mathcal{C}} WMC_c = \sum_{c\in\mathcal{C}} WMC^E_c \tag{2.31}$$

Since the degenerate hard c-partition space is a subset of the degenerate fuzzy c-partition space [19], we know that the set of all possible M is a subset of all possible P. Therefore by definition some possible P, i.e. those that are also a M, are correct when used for TD analysis. An evenly distributed P, where each entry is $\frac{1}{|\mathcal{C}|}$, is not useful since it would provide the same TD results for all objects. Obviously, the closer the estimated P is to a M that accurately represents the project's source code, the better the TD analysis will be. However, overfitting the estimate simply to be closer to the hard c-partition space could result in an inaccurate analysis. We discuss how finding appropriate fuzzy estimate sets for

P is important to provide a similarly useful TD analysis as analysis using a M. We propose the procedure for determining an appropriate fuzzy estimates would use a DR procedure or a combination of them.

Most of the DR discussed in the Design Recovery and Migration section include clustering the functions with objects such that a function has a membership to one object. That restriction is necessary when migrating code from PL to OO. We discuss that relaxing requirements in some of the DR algorithms, like some of the DR research suggest, will determine the fuzzy estimate needed to define a P.

The DR research using fuzzy estimates typically require a developer to interpret the results and determine the true membership during a migration. We discuss that the TD analysis can be performed with these fuzzy estimates since TD analysis results are designed to be interpreted by a developer already. The DR research, such as Livadas and Johnson [26], discuss the migration to be performed iteratively. That iterative process of refinement is typically how TD analysis results are used. The developer uses the TD analysis results to determine the order to prioritize refactors, but typically the developer will remove some TD and rerun TD analysis iteratively.

The mathematical foundation demonstrates the possibility of using a P if a DR algorithm can determine an appropriate P. This leads to our on going research to determine a DR algorithm to create P for TD analysis.

Significance of the Study

As discussed in Nelson and Izurieta (2021)[7], modern PL, such as Go, are being used heavily but do not have the same useful TD analysis tools as modern OO. The research on the impact of TD, such as [1, 3, 4, 5, 6], has determined that TD analysis identifies code that is slowing down development and costing companies' time and money. Refactoring to mitigate the TD has been proven to decrease TTD (time to deliver) and increase ROI

(return on investment). Companies will pay for tools, such as SonarQube and CodeScene. They typically integrate these tools into their development process, such as integrated into their CI/CD, because they find value in TD analysis. By providing a technique to perform TD analysis on PL to get similar results to the existing TD analysis on OO, companies can produce and maintain better PL projects faster and at reduced costs.

APPROACH

To test our hypothesis, we have been building a suite of tools. The tools includes a Go feature abstractor, a Java feature abstractor, a DR tool, a TD analysis tool, and a Test Automator. All of these tools are command line programs. For more information on how much has already been built, see Preliminary Work and Results. These tools are used together as shown in the software pipeline diagrammed in 3.1. The pipeline of tools is the result of the following steps. Most of the following steps incrementally add more to the pipeline so that we can slowly build up these tools and test the tools in the pipeline at each step. The pipeline allows us to quickly rerun tests and ensure correctness if any step requires us to modify any part of the tools that a prior step had already tested.

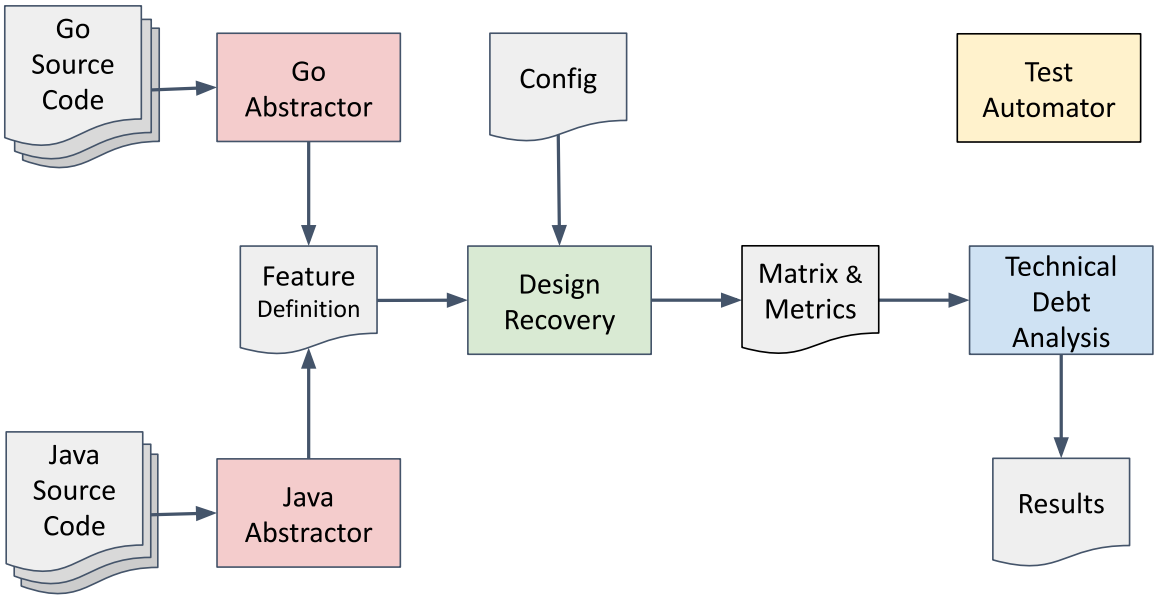


Figure 3.1: Software Pipeline Diagram

The sections in this chapter are in the order in which the steps of the approach have been and still need to be done for our research. The Determine Feature Description step determines the features that must be expressed in the feature definition file. The Java

Membership step prepares a large portion of the tools and ensures that we are performing TD analysis correctly. The Java Participation step determines the DR algorithm to create a participation matrix with the closest results to membership. The Identify Variance in Java step reviews the results of running TD analysis using a membership matrix and a participation matrix for the same Java projects to identify the causes of variance in the results. The Participation in Go step prepares the rest of the tools and ensures that we are getting reasonable results from Go projects. The Analyze Go Participation step uses our pipeline to test our hypothesis against real world conditions. The Report Findings step writes a report containing our findings. Finally, the Future Work step is optional research paths after our initial research is done.

Determine Feature Description

The first step we have to take is to determine what information is needed in the feature definition file. The feature definition file is a JSON file for a given source code similar to Zou and Kontogiannis' [34] AST conversion into XML. The feature definition will contain information about the object types and functions (i.e. classes and methods in OO) of a project. The feature definition must be language agnostic so that it can be used for Java and Go in such a way that the DR tool does not need to know the language of the project described in the file.

The feature definition must represent any membership found in the abstracted source code, including any defined Go receivers on functions. It must also store some metrics for a function, including the CC, function invocations, data reads, and data writes that are needed for TD analysis. Each identified feature (e.g. named object and named functions) must have the file path and line number that the code was defined on.

Since some of the DR needs information about a functions' signature, the fields of a structure, and the inheritance hierarchy of objects, those will need to be represented as well.

There may be some language specific information in the features, however these need to be limited so that the DR uses the same data flow when processing Go and Java projects. The language specific parts could be used to help with debugging.

Future researchers and practitioners will likely find it better to skip the feature definition file so that all supported features of a language can be taken into account when performing the DR. We use the feature definition file to ensure that the DR algorithm we have tested and chosen with Java will be the same as the one used for Go. By using the same code for both, we remove a threat to validity that would be introduced by having multiple copies of the DR algorithm. The more code that is shared between analyzing Go and analyzing Java, the more confident we can be that the Go and Java analysis will have similar results. This is the same reason we use the same TD analysis code.

The result of this step will be a description of the JSON file. This description may be modified if additional information is needed, however, we want to minimize how often we need to modify the description. Any change made will require updating the Go abstractor, the Java abstractor, and the DR tools, since the feature definition file is the communication between those tools.

Java Membership

This step is to write and ensure that we wrote our TD analysis correctly and ensure that the Java pathway in the tool pipeline has been prepared correctly as shown in diagram 3.1. The Test Automator needs to call the Java abstractor to get the feature definition file, pass the feature definition file into a basic form of the DR tool to get the Matrix and Metrics file (MAM), then pass that into the TD analysis tool to get the results.

The Java abstractor tool takes a path to a Java project to write a feature description

file. This tool is written in Java so that it can use Spoon[1] to parse the Java project into an AST.

The DR tool reads a feature description file and a configuration file. The configuration file is a JSON file with at least one field to select the algorithm to use. During this step, no DR algorithms will need to be defined. The configuration file will tell the DR tool to bypass DR such that it simply creates a membership matrix with any membership information from the feature description file. If this bypass algorithm was used on a feature description file from a Go project where membership information is only given for functions with receivers, the results might be similar to the results seen in the GodExpo section.

The TD analysis tool reads the MAM. The MAM contains a degenerate fuzzy c-partition space matrix, $m \in \mathcal{P}_{fo}$, as defined in 2.10. Since Bezdek and Harris show that $\mathcal{P}_{co} \subset \mathcal{P}_{fo}$, we know that we can store either a participation or membership matrix in the MAM. The metrics in the file will include the CC of each function, the function's name, file path, and line number. The metrics in the file will also include information for objects, including the object's name, file path, and line number. The file must provide enough information for the TD tool to calculate the WMC, TCC, and ATFD for each object. Then the tool will perform the Marinescu GC detection [2].

We use the Technical Debt Dataset (TDD)[2] [44] as the ground truth from other existing analysis tools. Given a version of a project, we will read the actual WMC, TCC, and ATFD from the TDD and compare the results against our calculated results. We expect to get identical results within some minor error allowance.

Since there is a specific number of Java projects in the TDD, we will manually download each at the correct version. We will also manually collect the expected result from the TDD via a database reader. None of these manual actions are depicted in the pipeline diagram.

[1] `https://spoon.gforge.inria.fr/`
[2] `https://github.com/clowee/The-Technical-Debt-Dataset`

We will prepare the Test Automator tool with a script that contains a list of the Java programs, their paths, and the expected results. This script can be run and rerun as many times as needed. For each project it will run the abstractor on the project's path to get the feature description, then pass that feature description into the DR tool with a pre-written configuration to bypass DR and simply return a MAM with the membership matrix. The resulting MAM is passed into the TD tool and the resulting TD analysis and metrics will be written to a results file.

The results from the pipeline and the ground truth values from the TDD will be used to calculate the mean squared error (MSE) using the Pearson Correlation Coefficient (PCC) for each of the results per project. The comparison results are the output we can use to determine if the pipeline and our TD analysis is working correctly. At each step, the files produced will be kept so that we can debug and retest parts of the pipeline in isolation until we have it working correctly.

Java Participation

Once the TD analysis using membership matrices is determined to work correctly, we will start trials on DR algorithms. We configure the DR tool to run different DR algorithms with different setups. Each algorithm will be setup via the configuration file so that several setups can be created and reused easily. This step will use the Java pathway through the pipeline, shown in diagram 3.1, that was prepared by the prior step.

Most algorithms should ignore the membership defined in the feature definition as if the source code did not describe the membership, even though we know that Java does define it. Some may use a subset of the membership information similar to the frequency with which Go receivers are defined, and the membership may be weighted as part of the DR algorithm. If membership is used too heavily here, the algorithms will work fine for Java but then function poorly for Go, so the use of them must be minimized.

The same sample selection of Java projects from the prior step will be used in this step so that we can compare the results from the participation matrix against the membership matrix knowing that the membership matrix is correct. With this pipeline, we can setup an experiment using different configurations for different DR algorithms. For the experiment to be run consistently, each DR algorithm must deterministically return the same participation matrix for the same input and configuration. Each algorithm and configuration pair is a treatment applied to the sample selection of projects. The resulting WMC, TCC, and ATFD are the dependent variables collected for all objects in each project.

For DR, objects represent groups of data written together by the developer of the source code, i.e. the classes in a Java project. Some of the DR algorithms will join data together to create an object, however, we do not need to join data since we can simply apply the membership proportionality to the individual data. For example, imagine two objects that are used together often enough in functions that the DR algorithm places both into the same cluster. We will then say that the functions in that cluster are half members of both objects. We do this so that, regardless of the DR algorithm, we will result in the same number of objects in the results, i.e. the participation matrices and membership matrix should all be the same size. This constraint is necessary for our results to be comparable against the ground truth, i.e. the results when using the membership matrix. However, future research may experiment with partial clustering and joining of objects to potentially produce better TD analysis.

The Test Automator will have a new script written that is similar to the prior step's script except it will run the pipeline with different configuration files. The script may even write the configuration with specific values for ranges we would like to test if needed. With the results of the experiment and the ground truth values, we calculate the MSE using the PCC for each of the results per project. With the MSE values, we can determine the DR algorithm that produces the most similar results to the membership matrix for the most

projects, i.e. the PCC closest to one on average for all three metrics across all projects.

Since the DR algorithms were created for different purposes than to produce participation matrices, this refinement will help to find the best configurations with the selected DR algorithm to produce participation matrices from code feature definitions. Obviously, this will only optimize the output for the Java samples and may not produce the best results for all Java nor Go projects. This is a known threat to validity, but accepted for reasons discussed in the Identify Variance in Java section. The goal of this step is to select the DR algorithm and settings that we will continue to use for the remaining steps.

Identify Variance in Java

It is unlikely that the DR algorithm will produce a participation matrix that is nearly identical to the membership matrix. There are several reasons that the classes might be clustered differently. In this step we will manually investigate the differences between the best results from the previous step and the ground truth.

The developer may have put a method as a member of a class which the method would not cluster with based on how the method is used. In many cases this is done on purpose to group creation methods, such as a class for creating system colors or a math package. In several cases, these are static methods in a static class, but not always. The factory pattern described in the Gang of Four book [37] is a non-static class that should match an interface and may be simply a wrapper around another class's constructor. The clustering algorithm may see that the return type of the factory method and invocation of the constructor inside the factory method are both for a class other than the factory class. Therefore, the factory method could be a full member with the class it generates and not a member of the factory class. The same could happen for other patterns, such as the bridge and adapter patterns.

The DR algorithm we choose will not be the best out of all possible algorithms, only the best from the few we implement whilst trying to get close to a PCC of one. During the

discussion after presenting our mathematical foundation paper [43], we got questions about possibly training an AI to read comments and read design documents to produce clusters closer to those intended by the developer. We will leave experimenting with different DR algorithms to find the best algorithm for a specific language to future researchers, as discussed in the Future Work section. We simply need any algorithm that is good enough to prove that TD analysis designed for OO can be run on PL using a DR to produce reasonable results. The algorithm we choose has to be language agnostic, or at minimum, it should only support features that exist in both Java and Go, i.e. works on the shared features chosen to be in the feature definition file.

We understand that choosing a DR algorithm that maximizes the PCC for a sample set of Java projects may mean that this algorithm is less than ideal for other Java and Go projects. This is acceptable so that we can validate our unproven ideas against proven results at each step. The reason for creating a pipeline is so that we can reduce our manual actions, allowing us to run trails consistently and rerun trails if we determine the DR algorithm does not perform well. The pipeline allows us to quickly iterate and retest changes to the algorithm on both Java and Go projects as needed.

Additionally, in OOs, any discrepancies could be used as an indicator that some patterns are being misused or that the code has less than ideal memberships, as long as the developers are aware of false positives. The differences could indicate that a method has been made a member of a less than ideal object based on the objects being used in that method. With some refinement, differences could be used as their own analytic to help determine more accurate pattern and anti-pattern matching, or even to identify a new kind of TD analytic suggesting a more ideal membership. However, such investigations are outside of the current body of research and are available for future research, as discussed in the Future Work section.

The goal of this step is to manually identify the causes for the results when using

participation to not align with the results when using membership. This will require us to read through the source code of the Java projects to see why some methods are throught to participate or not with a specific Java class. If the identified cause is a problem in the DR algorithm and setup, or something that can be fixed by adjusting the algorithm, then iterate these steps to remove those causes. Otherwise, the identified causes need to be recorded and kept as part of the threats to validity and known issues with the current approach. When reviewing the results for Go projects, these causes should be taken into account by the person doing the review, as discussed in Participation in Go. These identified causes might be investigated in future research when producing a better, more language specific, DR algorithm.

Participation in Go

Once a DR algorithm has been chosen, iterated on, and analyzed for OO, we can switch to working with PLs, specifically Go. This step will use the same DR and TD tools as the prior steps. The Go abstractor will be used for this step, as seen as the Go pathway through diagram 3.1. If any modification of those two tools is made, then the prior steps must be repeated to ensure that the tools still function correctly for Java. As stated before, the DR algorithm may not be the most ideal for Go, however, for this research the DR must remain consistent between Go and Java to reduce variability and noise.

The Go abstractor tool takes a path to a Go project, parses the project into an AST, and then converts the AST into a feature description file. Since Go's standard library contains a Go parser that provides an AST and type information[3], this tool is written in Go.

Go does not explicitly define membership in the source code. It does allow receivers to be added to functions to facilitate duck-typing. The receivers in many ways are similar to

[3] `https://pkg.go.dev/go/ast`

defining membership, but have some major differences, as discussed in [43]. Since receivers are optional on functions and act as an initial parameter not reflected in the function's signature, for now, the implied membership may be ignored or partially used by the chosen DR algorithm.

Some of the constructs in Go will not be useful by the DR, so some conversion to bring Go closer to Java will be necessary. Since some DR algorithms need inheritance, the duck-typing of Go's objects to interfaces can be resolved to determine an equivalent inheritance for a Go project.

The goal of this step is to get the Go abstractor working. This should be tested on several open source Go projects, including those discussed in [15], e.g. Hugo, and the TD results should be used to determine if they are reasonable for the Go project. When manually analyzing the results by reading through the tested Go projects, we needs to take into consideration the causes for variance found in the Java projects, such as Factory patterns.

Analyze Go Participation

Using the steps above, TD analysis result for Go projects can be calculated. Since Go does not explicitly define membership, there is no similar ground truth to the ones used in Java, therefore the results using participation can not be compared against the missing membership for the same source code. Instead, an empirical test will be performed to determine the validity of the results.

Find several Go projects from companies willing to participate in an empirical study. Calculate the TD analysis results for each project and determine if that aligns with the developers' perceived TD in that project.

The TD analysis results could be presented as a list of the top ten most likely GCs in the project. The developers indicate if that list contains the GCs they would expect, any

GCs that are missing, and anything that they would not consider to be a GC. The developers will have to understand that Go packages that are known junk drawers may not show up in this list since the functions may all have different participating objects. The developers will have the causes of variance explained to them so that they can take those into consideration when reviewing the results.

Alternatively, a defect detection tool, such as SZZ [24], could be run on public Go projects. The defects found by the tool that were fixed by a refactor could be used to test our PL TD analysis. Refactors tend to reduce GCs [5]. The TD analysis from the commit containing the reported defect and the commit containing the fix can be compared to determine if any of the GCs from the prior version have been removed. Since defect detection uses tickets and issues written by developers, these defects are self-reported. This process for testing Go participation may be more fault prone because of the additional work required and therefore will only be done if we cannot find at least three companies for an empirical test. If this process of testing is not done, then it could be a future research objective.

Report Findings

Once an empirical test has been performed. The findings of all steps should be reported. The chosen DR algorithm, causes of variance in Java, and results from Go should be discussed. This report should contain all threats to validity and define future research. This report will be the defense of this research and the conclusion of this proposal.

Future Work

As was stated throughout this report, there are many opportunities to expand on this work that is beyond the scope of this research. The following are potential directions that

we or other researchers may investigate.

Research should be done to repeat this work with different OOs than Java and PLs than Go. As seen in figure 1.1, another PL that could be investigated is Rust[4]. Additional research could be done to repeat this work with different software patterns and anti-patterns other than GCs to determine the usefulness of using participation in other TD analysis results.

The DR algorithm can be expanded to include more information from the source code and refined to a better solution specific to one language. Research could be done on some of the causes of variance between the membership and participation to determine whether it is possible to detect common patterns that are written correctly according to the developers, but cause the DR algorithm to cluster the methods differently. If those patterns could be detected, then the DR algorithm could be modified to take that information into account when creating the participation estimates. Additional research could be done on the use of a natural language processor or semantic analysis of code comments and additional documentation from a project as additional information to guide the DR algorithm.

For OO, a novel TD analytic could be studied that compares the participation from DR against the membership from the source code to suggest methods that are likely members of the wrong class and recommend the class that the method would more likely cluster with. This research could help provide suggestions to developers that could reduce ATFD and increase TCC to preemptively prevent GCs from being formed.

Lastly, research could be done on generating design diagrams, possibly via a Unified Modeling Language (UML), by further clustering the DR results, as discussed in Biggerstaff [21]. If the source code provides membership or the clustering leaves fuzzy estimates, the design diagram could show the variance as some kind of difference indicator between two patterns. This kind of visual feedback could help practitioners onboard new developers and

[4] `https://rust-lang.org/`

better understand their systems.

PRELIMINARY WORK AND RESULTS

The following is a breakdown of each of the components and how much of the preliminary work we have accomplished so far. The code and documentation for this work is in the GitHub repository `msusel-tdmetrics-go`[1]. In addition to the work defined below and our prior published works [7, 43], we have downloaded the TDD and determined how to read the table information from the flat database file, we have setup GitHub Actions (GHA) on our repository to automatically run the unit-tests, linting, and quality testing on each commit, and we have investigated the validity of GodExpo [15] code and Sarker, Jubaer, Shohrawardi, Das, and Siddik [16], discussed in the Background.

Feature Definition

We have determined the features that need to be defined in the feature definition file. The repo contains a detailed definition[2]. The overall design is to have a project contain lists of different types of constructs. Some of the constructs are methods, basic types, objects, signatures, and structures. All constructs are defined at this top level without nesting.

A construct can reference another construct. For example a structure has a list of references for the fields in the structure, and a signature has a list of references for parameter name and type pairs and another list for result name and type pairs. The references are either an index if there can be only one construct list that can be used for that reference or a construct name and index when the constructs could come from multiple lists. The reason for having the constructs defined like this is because many types will have circular definitions. For example a node in a linked list will have a next value field of the same type of node as itself. By using indexed references and flat lists for all the constructs, the file reader

[1] `https://github.com/MSUSEL/msusel-tdmetrics-go`
[2] `https://github.com/MSUSEL/msusel-tdmetrics-go/blob/main/docs/genFeatureDef.md`

can simply create stubs for all the constructs in each list, then as the stubs get populated, the reference can easily look up a construct that may or may not have been populated yet. Since Go defaults values to zero, we decided that our indices will be one-based so that we can quickly differentiate between unset and the first index.

Go may have multiple optionally named results types on a signature and Java has a single unnamed return type, therefore we made the signature accept multiples results with optional names. That allows Java to simply put one unnamed result type per signature. Otherwise we would have to create structures to represent a return type with multiple results for Go. Since the *tr* in TBOI, discussed in the Livadas and Johnson section, puts no constraint on the number of result types allowed, there is no indication having multiple results types would be problematic.

Go allows named types that are not structures nor interfaces, e.g. `type X []int`. Those types would have to be represented in some way. We decided that for those cases we treat them as structures with a single field of that type, e.g. `type X struct { $data []int }`. Java classes can be treated like a named structure with fields where the methods have the class as the receiver. By converting types in this way, we will not have the same problems as GodExpo, discussed in the GodExpo subsection, since all types, not just those written with underlying structures, will be taken into account.

To provide the file path and line number for methods and types, a map is added to the features. The map has an offset number as the key to a file path value. The offsets are the accumulated lines numbers such that the first file has the offset of one, and if that file has n lines, then the second file with have the offset n. The methods and types have a location file that is a single integer. That location is the offset to the file path and line number, such that if the location is $n + 4$, then that location is in the second file on the fifth line.

Go is a duck-typed language without inheritance, so we will have to determine type inheritance using the duck-typing. This inheritance is based off of the super-type and sub-

type definitions described in Liskov and Wing [13].

The feature definition defined in the repo has worked for our research so far and is fairly descriptive. Since the file is JSON, additional information can be added to help with debugging, such as scoping. Tests can also use a subset of YAML that maps to JSON so that comments can be added and short hand for JSON objects can be used for readability. We have coded and tested the feature definition writer for the Go and Java abstractor. Both the Go and Java abstractor have simple feature definition reader for testing. We have coded and tested the feature definition reader for the `C#` DR tool.

Go Feature Abstractor

To ensure that the feature definition file could be used for Go, we created the Go feature abstractor already. Go abstraction was more of a concern since abstracting features from OO, such as Java, is defined in papers such as [34] already. We had some difficulty creating the abstractor because we started working on this before the full release of generic types in Go. Generics were added in Go v1.18 in March 2022. Since the Go abstractor may be run on unknown Go projects when performing the empirical test, we have keep our abstractor up-to-date with the current Go version since it is likely that the companies will keep their projects up-to-date and may used the latest Go features. The Go abstractor is currently at Go v1.25.1.

We had to rework our abstractor to handle generic types and type instantiation. Our feature definition file already could handle generics like Java has by design. Unfortunately, Go's generics included type constraints which complicated the plan we had for generics. Generics greatly compounded type determination since now there were nested types, such as a concrete type inside of a generic function definition. The nested types are not well supported by Go's type information system when reading from the AST.

Ultimately, we have a functional Go abstractor that will work for our research, however

it is far more complicated than it needs to be since we did not do a full redesign when including generics. Practitioners using our research should be aware that the Go abstractor could be refactored to simplify much of the abstraction process. One complication that is likely not needed is that we wrote our Go code to control the mutability of the construct objects being abstracted so that large sets of data can be quickly sorted and looked up, and to reduce possible bugs from being introduced. The abstractor could be written without this control and instead use fully mutable structures with type switching to handle polymorphism instead of shared interfaces.

To test that our Go abstractor is functional, we have written several integration tests that abstract small Go projects and checks the result against an expected feature definition file.

Java Feature Abstractor

The Java feature abstractor has had enough functionality added to it that it can be run on basic examples. Since Java has far more language complexity than Go we will not implement a full Java abstractor for all constructs in Java. We only need the constructs that are used in the projects that the TDD has recorded metrics for.

We investigated several of the different tools for reading Java code into an AST or similar queryable structure. This included the Apache ClassReader[3], JavaParser[4], and Spoon [45][5]. Our current progress has shown that Spoon will likely work for all the constructs that we need to handle.

The Java abstractor is built to work as similarly to how the Go abstractor works so that both can be run similarly by the Test Automator via command line. To run abstraction

[3] `https://axis.apache.org/axis/java/apiDocs/org/apache/axis/utils/bytecode`
[4] `https://github.com/javaparser/javaparser`
[5] `https://spoon.gforge.inria.fr/`

on the projects from TDD we had to use OpenJDK 17.0.14 for Java 17 with Apache Maven v3.9.9.

The work on the Java feature abstractor will continue as outlined in the Approach. See the Timeline for more information.

Design Recovery and Technical Debt Analysis

To ensure that there is no bias intrinsic in either Go and Java when performing DR and TD analysis, the DR tool and TD analysis tool are being written in `C#`. To simplify the research, the Test Automator and TDD flat file database reader have also been written in `C#`. Each of the different tools are built and run separately but may share libraries.

The feature definition file reader has been implemented as well as tools to read and write configuration files, and MAM files. Since the matrices for a full project could get very large but contain a lot of zero values, the matrix tools and serialization implements a simple sparse matrix design via a bipartite graph that takes into account the requirements for the participation matrices.

Currently the DR tool simply reads the feature definition file and then either presents the information in a simple to read human form for quick iteration and testing, or writes a MAM for membership, as discussed in the Java Membership section. The work on these tools will continue as outlined in the Approach.

TIMELINE

The following outlines the timeline for the completion of the work described in the Approach. The aggressiveness of this timeline is to depict the ideal path. This will give us time to expand our timeline if any complication is encountered, such as a company that agreed to an empirical test need us to wait until later than expected to perform the test or they back out requiring us to find another company.

4th Quarter 2025

- The first priority is to present this Comprehensive work and, assuming that this work is acceptable, pass the Comprehensive exam.
- Finish the Java abstractor, finish the TD tool, and test the basic DR tool, to confirm that the Java abstractor and the TD tool are working correctly. As discussed in the Java Membership section, the goal is to confirm that we can get the same results as those in the TDD.

1st Quarter 2026

- Start contacting companies that may be willing to participate in our empirical test with their Go code and developers.
- Finish the DR tool with our implementation of a modified Livadas and Johnson [26] DR algorithm.
- Start comparing participation matrices against membership matrices for Java as described in the Java Participation section.

- Start identifying the variance between membership and participation results, as described in the Identify Variance in Java section.

2nd Quarter 2026

- Finish comparing participation matrices and identifying variance. Determine whether the current approach is going to work or if we have to rethink our approach, i.e. can participation work well enough with Java that we believe continuing onto Go at this point is reasonable.
- Update the Go abstractor for any new versions that have been released since the last time the Go abstractor was updated. Make sure the tool works as described in the Participation in Go section.
- Start testing the Go abstractor on various code bases to see if it appears to be working. All of these results will be antidotal. They are mainly used to ensure that the application does not crash when running the empirical tests so that we do not waste the companies' time.
- Start setting dates for meeting with companies to run the empirical tests.
- Start working on the questionnaire used to collect information from developers in relation to the empirical test.

3rd Quarter 2026

- Run the empirical test with the companies that agreed to run the test. If there were any issues running the test, collect some debug information and set a time to try again. Collect the test results with the questionnaire.

- Start analyzing the results of the empirical test and start writing a paper to report the results as described in the Report Findings section.

4th Quarter 2026

- Finish writing the paper with the results and present the findings. Upon this report being accepted and defended, this should fulfill the requirements for the degree of Doctor of Philosophy in Computer Science.

REFERENCES CITED

[1] Clemente Izurieta, Ipek Ozkaya, Carolyn Seaman, and Will Snipes. Technical debt: A research roadmap report on the eighth workshop on managing technical debt (mtd 2016). *ACM SIGSOFT Software Engineering Notes*, 42:28–31, 03 2017.

[2] Radu Marinescu. Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE international Conference on Software Maintence*, pages 350–359, ICSM. IEEE Computer Society, Washington, DC, 2004.

[3] Clemente Izurieta, Ipek Ozkaya, Carolyn Seaman, Philippe Kruchten, Robert Nord, Will Snipes, and Paris Avgeriou. Perspectives on managing technical debt: A transition point and roadmap from dagstuhl. In *International Workshop on Technical Debt Analytics (TDA)*, Hamilton, New Zealand, 2016.

[4] Alan MacCormack and Daniel J. Sturtevant. Technical debt and system architecture: The impact of coupling on defect-related activity. *J. Syst. Softw.*, 120:170–182, 2016.

[5] Nico Zazworka, Michele A. Shaw, Forrest Shull, and Carolyn Seaman. Investigating the impact of design debt on software quality. In *Proceedings of the 2nd Workshop on Managing Technical Debt*, volume MTD'10, 978-1-4503-0586-0/11/05, New York, NY, USA, 2011. Association for Computing Machinery.

[6] Neil A. Ernst, Stephany Bellomo, Ipek Ozkaya, Robert L. Nord, and Ian Gorton. Measure it? manage it? ignore it? software practitioners and technical debt. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, page 50–60, New York, NY, USA, 2015. Association for Computing Machinery.

[7] Grant Nelson and Clemente Izurieta. A gap in the analysis of technical debt in procedural languages: An experiential report on go. *IEEE Software*, 38(6):71–75, 2021.

[8] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.

[9] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[10] James M. Bieman and Byung-Kyoo Kang. Cohesion and reuse in an object-oriented system. In *SSR '95*, 1995.

[11] Frank Calliss and Barry Cornelius. Potpourri module detection. In *Proceedings. Conference on Software Maintenance 1990*, pages 46–51, 1990.

[12] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of ACM*, 15(12), 1972.

[13] Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, nov 1994.

[14] L.A. Belady and C.J. Evangelisti. System partitioning and its measure. *Journal of Systems and Software*, 2(1):23–29, 1981.

[15] Rafed Md. Yasir, Moumita Asad, Asadullah Galib, Kishan Ganguly, and Md Saeed Siddik. Godexpo: An automated god structure detection tool for golang. In *2019 IEEE/ACM 3rd International Workshop on Refactoring (IWoR)*, pages 47–50, 05 2019.

[16] Maloy Sarker, Abdullah Jubaer, Md. Shihab Shohrawardi, Tulshi Das, and Md. Saeed Siddik. Analysing golang projects' architecture using code metrics and code smell. In Sajid Anwar and Abdul Rauf, editors, *Proceedings of the First International Workshop on Intelligent Software Automation*, pages 53–63, Singapore, 2021. Springer Singapore.

[17] Moumita Asad, Rafed Md. Yasir, Shihab Shahriar, Nadia Nahar, and Md. Nurul Ahad Tawhid. Analyzing program comprehensibility of go projects. In *33rd International Conference on Software Engineering and Knowledge Engineering (SEKE)*, pages 255–260, 07 2021.

[18] Mika Mäntylä, Jari Vanhanen, and Casper Lassenius. A taxonomy and an initial empirical study of bad smells in code. In *International Conference on Software Maintenance, 2003. ICSM 2003. Proceedings.*, pages 381–384, 2003.

[19] James C Bezdek and J Douglas Harris. Fuzzy partitions and relations; an axiomatic basis for clustering. *Fuzzy Sets and Systems*, 1(2):111–127, 1978.

[20] Miin Shen Yang. A survey of fuzzy clustering. *Mathematical and Computer Modelling*, 18(11):1–16, 1993.

[21] Ted J. Biggerstaff. Design recovery for maintenance and reuse. *Computer*, 22(7):36–49, 1989.

[22] Elliot Chikofsky and James Cross. Reverse engineering and design recovery: a taxonomy. *IEEE Software*, 7(1):13–17, 1990.

[23] David H. Hutchens and Victor R. Basili. System structure analysis: Clustering with data bindings. *IEEE Transactions on Software Engineering*, SE-11(8):749–757, 1985.

[24] Markus Borg, Oscar Svensson, Kristian Berg, and Daniel Hansson. SZZ unleashed: An open implementation of the SZZ algorithm. *CoRR*, abs/1903.01742, 2019.

[25] Sying-Syang Liu and Norman Wilde. Identifying objects in a conventional procedural language: an example of data design recovery. In *Proceedings. Conference on Software Maintenance 1990*, pages 266–271, 1990.

[26] Panos E. Livadas and Theodore Johnson. A new approach to finding objects in programs. *J. Softw. Maintenance Res. Pract.*, 6:249–260, 1994.

[27] Harry Sneed and Erika Nyáry. Extracting object-oriented specification from procedurally oriented programs. In *Proceedings of 2nd Working Conference on Reverse Engineering*, pages 217–226, 1995.

[28] Aniello Cimitile, Andrea De Lucia, Giuseppe Antonio Di Lucca, and Anna Rita Fasolino. Identifying objects in legacy systems using design metrics. *Journal of Systems and Software*, 44(3):199–211, 1999.

[29] Sagar Pidaparthi and Grzegorz Cysewski. Case study in migration to object-oriented system structure using design transformation methods. In *Proceedings. First Euromicro Conference on Software Maintenance and Reengineering*, pages 128–135, 1997.

[30] Sagar Pidaparthi, Hussein Zedan, and Paul A. Luker. Resource usage matrix in object identification and design transformation of legacy procedural software, 1997.

[31] Sagar Pidaparthi, Hussein Zedan, and Paul A. Luker. Conceptual foundations for the design transformation of procedural software to object-oriented architecture, 1998.

[32] Néstor A. González, Chris Czarnecki, and Sagar Pidaparthi. Migrating software from procedural to object-oriented architecture. In *SMC'98 Conference Proceedings. 1998 IEEE International Conference on Systems, Man, and Cybernetics (Cat. No.98CH36218)*, volume 5, pages 4872–4877 vol.5, 1998.

[33] Sagar Pidaparthi, Paul A. Luker, and Hussein Zedan. 13 - reengineering procedural software to object-oriented software using design transformations and resource usage matrix. In H. Zedan and A. Cau, editors, *Object-Oriented Technology and Computing Systems Re-engineering*, pages 182–197. Woodhead Publishing, 1999.

[34] Ying Zou and Kostas Kontogiannis. A framework for migrating procedural code to object-oriented platforms. In *Proceedings of the Eighth Asia-Pacific on Software Engineering Conference*, APSEC '01, page 390, USA, 2001. IEEE Computer Society.

[35] Shahadat Hossain and Tahsin Zulkarnine. *Design structure of scientific software - A case study*, pages 129–134. Carl-Hanser-Verlag, Munich, 2011, Cambridge, Massachusetts, USA, 09 2011.

[36] Md. Saeed Siddik, Alim Ul Gias, and Shah Mostafa Khaled. Optimizing software design migration from structured programming to object oriented paradigm. In *16th Int'l Conf. Computer and Information Technology, ICCIT 2013*, 12 2013.

[37] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Pearson/Addison-Wesley, 1995.

[38] Md. Selim, Md. Saeed Siddik, Alim Ul Gias, M. Abdullah-Al-Wadud, and Shah Mostafa Khaled. A genetic algorithm for software design migration from structured to object oriented paradigm. *CoRR*, abs/1407.6116, 2014.

[39] Md. Saeed Siddik, Alim Ul Gias, Md. Selim, Shah Mostafa Khaled, and Kazi Sakib. A direction of migrating procedural paradigm to object based architecture by forming cluster of functions using local search heuristics. In *2014 International Conference on Informatics, Electronics & Vision (ICIEV)*, pages 1–6, 2014.

[40] Md. Saeed Siddik, Md. Selim, Alim Ul Gias, and Shah Mostafa Khaled. Migrating to object oriented architecture from procedural paradigm: A variable neighborhood search based approach. *Jahangirnagar University Journal of Information Technology*, 5:23–34, 07 2016.

[41] Mohayeminul R. Islam, Tajkia Rahman Toma, Md. Reza Selim, Alim Ul Gias, and Shah Mostafa Khaled. Design migration from procedural to object oriented paradigm by clustering data call graph. *International Journal of Information Engineering and Electronic Business*, 8:1–13, 2016.

[42] Md. Samsuddoha, Md. Saeed Siddik, Md. Selim, and Shah Mostafa Khaled. Empirical approach to validate heuristic algorithms for procedural programming to object oriented design migration. *International Journal of Advanced Research*, 5:2320–5407, 09 2017.

[43] Grant Nelson, Clemente Izurieta, and Derek Reimanis. Membership and participation in object oriented and procedural paradigms. *2023 Intermountain Engineering, Technology and Computing (IETC)*, pages 247–252, 2023.

[44] Valentina Lenarduzzi, Nyyti Saarimäki, and Davide Taibi. The technical debt dataset. In *Proceedings of the Fifteenth International Conference on Predictive Models and Data Analytics in Software Engineering*, PROMISE'19, page 2–11, New York, NY, USA, 2019. Association for Computing Machinery.

[45] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. Spoon: A Library for Implementing Analyses and Transformations of Java Source Code. *Software: Practice and Experience*, 46:1155–1179, 2015.